**Optimization Techniques For Matrix Multiplication**

(Thesis format: Monograph)

by

Md. Nazrul Islam

Graduate Program
in
Computer Science

A thesis submitted in partial fulfillment
of the requirements for the degree of
Master of Science

The School of Graduate and Postdoctoral Studies
The University of Western Ontario
London, Ontario, Canada

THE UNIVERSITY OF WESTERN ONTARIO
THE SCHOOL OF GRADUATE AND POSTDOCTORAL STUDIES

**CERTIFICATE OF EXAMINATION**

Supervisor:

Dr. Éric Schost

Examination committee:

Dr. Roberto Solis-Oba

Dr. Stephen Watt

Dr. Rob Corless

The thesis by

**Md. Nazrul Islam**

entitled:

**Optimization Techniques for Matrix Multiplication**

is accepted in partial fulfillment of the
requirements for the degree of
Master of Science

Date _____ Chair of the Thesis Examination Board

# Abstract

Matrix multiplication is a cornerstone of linear algebra algorithms; its asymptotic complexity, when the matrix' size tends to infinity, has attracted a lot of attention in the last forty years.

On the other side of the spectrum, the cost of operations on small dimension matrices remains imperfectly understood. However, small matrices, but with large entries, are used throughout many algorithms, such as the evaluation holonomic functions by means of the binary splitting algorithm, polynomial system solving, etc. Our objective is to shed some light on this problem, obtaining matrix multiplication procedures that reduce the number of multiplications required for such small matrices.

We base our work on an optimized usage of the previous algorithms for small matrix multiplications, due to Strassen, Winograd, Pan, Laderman, etc.; we show how to exploit all of these standard algorithms recursively in an improved way.

As a result, we were able to get lower numbers of multiplications for many matrix sizes between 1 and 30. We wrote a code generator that uses this table to generate multiplication code for long integers and polynomials, but also more complex data structures such as differential operators or linear recurrences.

**Keywords.** Matrix multiplication, Complexity, Partitioning, Optimization, Code generation.

# Acknowledgments

I would first like to thank my thesis supervisor Assistant Professor Éric Schost in the Department of Computer Science at University of western Ontario. The door to Prof. Éric Schost office was always open whenever I ran into a trouble spot or had a question about my research or writing. He consistently helped me on the way of this thesis and steered me in the right direction whenever he thought I needed it. I am grateful to him for his excellent support to me in all arenas.

Sincere thanks and appreciation are extended to all the members from our Ontario Research Centre for Computer Algebra (ORCCA) lab and the Computer Science Department for their invaluable teaching and assistance.

# Contents

# List of Algorithms

# List of Figures

# List of Tables

# Chapter 1

# Introduction

## 1.1 Matrix multiplication: a short brief

Multiplying matrices is one of the most basic operations for symbolic and numeric computing; on top of it, one can build algorithms for most other problems in linear algebra (matrix inversion, characteristic polynomial computation, etc), and beyond: matrix multiplication is used in algorithms for computing polynomial gcd's [11, Ch. 11], Padé-Hermite approximants [2], factoring polynomials [11, Ch. 14], evaluating holonomic functions [20], solving polynomial systems [24], etc. A few of these applications are reviewed at the end of this chapter.

To fix notation, if $A = [a_{i,j}]$ is an $m \times n$ matrix and $B = [b_{j,k}]$ is an $n \times p$ matrix, then the matrix product $C = AB$ is the $m \times p$ matrix $C = [c_{i,k}]$ defined by

$$c_{i,k} = \sum_{j=1}^{n} a_{i,j} b_{j,k} \quad 1 \leq i \leq m, \quad 1 \leq k \leq p.$$

Multiplying two square matrices having size $n \times n$ costs $n^3$ multiplications using the naive method. So, the naive approach to matrix multiplication involves $O(n^3)$ scalar operations on $O(n^2)$ data items. Developing ideas and algorithms to reduce this cost has been of the central problems in algebraic complexity since the 1960's.

## 1.2 Matrix multiplication exponent

The key event in the history of fast matrix multiplication algorithms is Strassen's discovery of an algorithm for multiplying $2 \times 2$ matrices using 7 (instead of 8) multi-

plications. Given matrices

$$A = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}, \quad B = \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix},$$

we are to compute their product

$$C = \begin{bmatrix} c_{1,1} & c_{1,2} \\ c_{2,1} & c_{2,2} \end{bmatrix}.$$

Strassen's algorithm computes the 7 products

- $\gamma_1 = a_{1,1}b_{1,1}$
- $\gamma_2 = a_{1,2}b_{2,1}$
- $\gamma_3 = (a_{1,1} + a_{1,2} - a_{2,1} - a_{2,2})b_{2,2}$
- $\gamma_4 = a_{2,2}(b_{1,1} - b_{1,2} - b_{2,1} + b_{2,2})$
- $\gamma_5 = (a_{2,1} + a_{2,2})(-b_{1,1} + b_{1,2})$
- $\gamma_6 = (a_{2,1} + a_{2,2} - a_{1,1})(b_{2,2} - b_{1,2} + b_{1,1})$
- $\gamma_7 = (a_{1,1} - a_{2,1})(-b_{1,2} + b_{2,2})$

and obtains the entries of $C$ as

- $c_{1,1} = \gamma_1 + \gamma_2$
- $c_{1,2} = \gamma_1 + \gamma_3 + \gamma_5 + \gamma_6$
- $c_{2,1} = \gamma_1 - \gamma_4 + \gamma_6 + \gamma_7$
- $c_{2,2} = \gamma_1 + \gamma_5 + \gamma_6 + \gamma_7$.

Using block matrix multiplication (which we explain in Section 2.2.2), it is possible to multiply $4 \times 4$ matrices using 49 multiplications, and more generally $2^k \times 2^k$ matrices using $7^k$ multiplications. If the size of the input matrices is not a power of 2, we can *pad* them with zero rows and zero columns, or *peel* then from a few rows or columns (that are treated separately), to obtain a size which is a power of 2. In any case, it is then possible to multiply $n \times n$ matrices using $O(n^{\log_2(7)})$ multiplications. The total number of operations (including additions as well) is $O(n^{\log_2(7)})$ as well.

The number $\omega = \log_2(7) \approx 2.807$ is called the *exponent* of Strassen's algorithm; it is to be compared to the exponent of the naive algorithm, which is 3. The key question regarding the complexity of matrix multiplication is whether there exists an algorithm of exponent 2, which would be optimal. This question is still open.

All matrix multiplication algorithms are roughly built on the same approach as Strassen's. First an algorithm for a specific small matrix problem is developed; then a block construction produces from it an algorithm for multiplying large matrices. For the last forty years, the ground rules for constructing basic algorithms and block constructions have been relaxed: the more advanced constructions involve sparse matrices, the introduction of power series coefficients (so-called *$\varepsilon$-algorithms*), etc.

Historically, the key question was mainly to reduce the exponent. The following list gives an (incomplete) overview of a series of improvements.

- Strassen's algorithm [27], found in 1969, has an exponent $\omega = \log_2(7) \approx 2.807$.
- Bini, Capovani, Romani and Lotti's $\varepsilon$-algorithm (1979) has $\omega = 3\log_{12}(10) \approx 2.779$ [3]; it uses the product of $2\times2$ triangular matrices by $2\times2$ square matrices as a base case.
- Using the same construction, Schönhage's $\tau$-theorem [23] (1981) gives an exponent $\omega = 3\log_6(5) \approx 2.694$. Using more a complex construction, Schönhage reduced the exponent to 2.548, and a construction of Pan further reduced the exponent to 2.522.
- Strassen's *laser method* (1987) has an exponent of 2.48 [28].
- In 1990, Coppersmith and Winograd [10] improved on Strassen's laser construction and gave an algorithm with $\omega \leq 2.376$; this is the best result to date.
- Recently, using the group-theoretic approach introduced in [9], Cohn, Kleinberg, Szegedy and Umans [8] recovered the exponent $\omega \leq 2.376$.

However, as the improvement over the exponent $\omega$ went on, the hidden constant in the big-Oh estimate grew dramatically, and the practical implementation of algorithms with such low exponents became extremely complicated. As a matter of fact, *none* of the previous methods is better than Strassen's original design for matrices of size less than several thousands of billions.

There exists however another line of work, which gave *practical* algorithms for matrices of more reasonable sizes, even though these results usually did not improve on the best known exponents.

- Laderman [16] gave in 1975 an algorithm for $3 \times 3$ products using 23 multiplications, whose exponent is $\omega = \log_3(23) \approx 2.854$;
- Makarov [19] gave in 1987 an algorithm for $5 \times 5$ products using 100 multiplications, whose exponent is $\omega = \log_5(100) \approx 2.862$;
- Pan [21] designed algorithms to perform two, or three, disjoint multiplications simultaneously, and showed how to adapt them to perform single products. These ideas were put to practice by Laderman, Pan and Sha [17] and Kaporin [14, 15]; the best previously known algorithm in this family features an exponent $\omega \leq \log_{44}(36388) \approx 2.776$ (we will slightly improve it in Chapter 5).

These techniques, and a few other ones, will be presented in more detail in Chapter 3.

## 1.3 Our considerations: small matrices

The algorithmic and software techniques used to speed-up matrix multiplication vary according to the size and nature of matrices considered. For instance, for large matrices with `double` coefficients, optimizing memory accesses is at least as important as reducing the operation count, and the cost of addition and multiplication may be comparable.

The focus of this thesis is at the opposite of the spectrum: we want to multiply efficiently small matrices with large entries (e.g., multiprecision integers, high-degree polynomials). In such cases, multiplication takes much more time than addition or subtraction, and the primary target is to reduce the number of multiplications. We detail one such example, and mention more briefly a few other ones.

**Evaluation of holonomic functions.** As an example, consider the high-precision evaluation of holonomic functions, that is, functions $f(x)$ that satisfy a linear differential equation with polynomial coefficients. From the differential equation, it is possible to deduce a recurrence satisfied by the power series coefficients $f(x) = \sum_{i \geq 0} f_i x^i$ (assuming this development is well-defined). As a result, we can use the recurrence to compute the values of $f$.

To do so, a classical approach is the binary splitting algorithm . We illustrate this algorithm with $f(x) = \exp(x)$. To evaluate the exponential at 1, we consider the sequence

$$e_n = \sum_{k=1}^{n} \frac{1}{k!},$$

whose limit is exp(1). We get the recurrence

$$(n+1)(e_{n+1} - e_n) = e_n - e_{n-1}$$

which becomes

$$\begin{bmatrix} e_{n+1} \\ e_n \end{bmatrix} = \frac{1}{n+1} \begin{bmatrix} n+2 & -1 \\ n+1 & 0 \end{bmatrix} \begin{bmatrix} e_n \\ e_{n-1} \end{bmatrix}$$
$$= A_n \begin{bmatrix} e_n \\ e_{n-1} \end{bmatrix}$$

where

$$A_n = \frac{1}{n+1} A'_n$$

and

$$A'_n = \begin{bmatrix} n+2 & -1 \\ n+1 & 0 \end{bmatrix}.$$

To compute $e_n$, we actually compute $\frac{1}{n!} A'_n \cdots A'_1$, since the value of $e_n$ is the entry at position $(1,1)$ in this product. To obtain an efficient algorithm, we use a binary tree structured computation. At the beginning, many matrices with small entries are listed; after the first level of computation, the entries within the matrices have grown larger, and so on. At the root of the tree we have two matrices with large entries (about $n \log(n)$ digits).

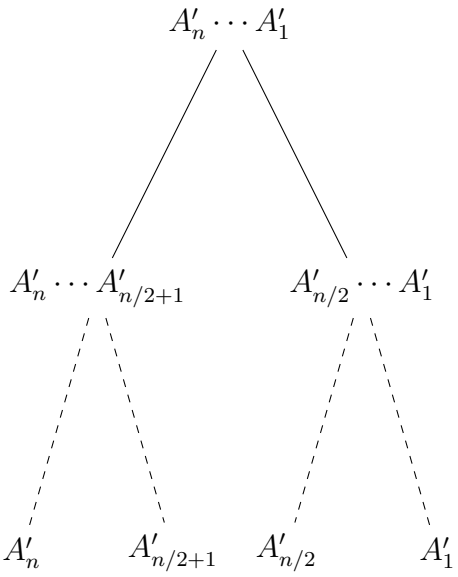



Figure 1.1: Evaluation of exp(1)

On this example, Strassen's algorithm allows us to perform a $2 \times 2$ product using 7 multiplications instead of 8. In general, the size of the matrices are equal to the order of the recurrence, which is also roughly equal to the degree of the coefficients appearing in the differential equation satisfied by $f(x)$; hence, sizes of size up to 10 or 20 are already "large" for such applications.

**Other applications.** To conclude, we mention a few other situations where such products of small matrices with large entries arise:

- *Padé-Hermite approximation.* Given power series $f_0(x), \ldots, f_m(x)$, Padé-Hermite approximation seeks polynomial coefficients $a_0(x), \ldots, a_m(x)$ of prescribed degrees, such that

$$a_0(x)f_0(x) + \cdots + a_m(x)f_m(x) = 0 \bmod x^\sigma,$$

  where $\sigma$ is usually chosen close to $\sum_i \deg(a_i)$. For instance, given the series expansion $1+x/2-x^2/8+x^3/16-5x^4/128+\cdots$ of $f = \sqrt{1+x}$, this allows one to recover the relation $(1+x) - f^2 = 0$, by taking $f_0 = 1, f_1 = f, f_2 = f^2$. One of the best algorithms for this task is due to Beckermann and Labahn [2], and follows a divide-and-conquer approach; it requires to multiply matrices with large polynomial entries.

- *Lifting techniques.* Triangular representations are a versatile data structure for solving polynomial systems of equations. A useful algorithm for this data structure is a *lifting algorithm* [24] which enables one to start from a representation with coefficients known modulo a prime $p$ and deduce "better" representations with coefficients known modulo powers of $p$. Here, given the polynomials

$$\left| \begin{array}{rcl} T_{1,0} & = & x_1^2 + 64x_1x_2^2 + 99x_1x_2 + 21x_1 + 8x_2^2 + 20x_2 + 17, \\ T_{2,0} & = & x_2^3 + 15x_2^2 + 64x_2 + 28 \end{array} \right.$$

  with coefficients defined modulo 101, the lifting algorithm will produce the polynomials

$$\left| \begin{array}{rcl} T_{1,1} & = & x_1^2 + (1637 + 1311x_2 + 1276x_2^2)x_1 + 1220x_2^2 + 1434x_2 + 1027, \\ T_{2,1} & = & x_2^3 + 1025x_2^2 + 1680x_2 + 1644 \end{array} \right.$$

  with coefficients defined modulo $101^2 = 10201$. As it turns out, one of the main operations in this algorithm is the product of matrices, whose elements

are multivariate polynomials as above. Their size is equal to the number of variables in the system we want to solve, so usually of order 10 to 20 at most.

## 1.4 Overview of the thesis and of our contributions

In this thesis, we focus on small matrices, typically from size 2 to 30, and try to find algorithms with the smallest possible number of multiplications (we will pay no attention to the number of additions, subtractions, or more generally multiplications by constants).

Even for small size matrices, improvement is not straightforward: such problems sizes have already been investigated thoroughly, and tables listing the best know multiplication counts have already appeared in [22, 26, 20]. This question is all the more intriguing as the known lower bounds are rather far from the upper bounds: the best lower bound is $2.5n^2 - 3n$ for multiplication in size $n$ [4, 5].

Our strategy (already used in the references above) is to combine previously known approaches (such as Strassen's, Laderman's and Makarov's, that were mentioned before), in an optimized and automated manner by finding the best fit for any given problem size.

Chapter 2 introduces background material, basic definitions, and some useful notation used in this thesis. Chapter 3 then presents previous work about matrix multiplication algorithms; it is out of the scope to give an exhaustive description of the vast amount of previous work; we only details for algorithms that we will use. The next three chapters give our main algorithm contributions:

- Chapter 4 describes an alternative to the padding or peeling techniques, useful e.g. when trying to apply Strassen's algorithm on matrices of odd size.
- Chapter 5 gives an improvement of an algorithm by Pan for performing three disjoint products, and its extension to perform a single product.
- Chapter 6 provides a generalization of an algorithm by Waksman for matrix multiplication with commutative entries, to the case of rectangular matrices.

Combining these results, we show in Chapter 7 how to build in an automated manner a database of matrix multiplication algorithms; we provide results up to size 30, and show that they improve previous ones for most sizes.

These results are not only of a theoretic nature. In Chapter 8, we present our work on code generation for a matrix multiplication library. Using the results of our

search, we produce implementations of the corresponding multiplication algorithms. The generated code is written in C or C++, and is based on the libraries GMP [1] for multiprecision integer multiplication, and NTL [25] for polynomial arithmetic. Finally, Chapter 9 presents our conclusions and future work.

# Chapter 2

# Basics

The first aim of this chapter is to provide the background and notation used in this thesis for matrix operations. We start by recalling basic definitions, properties, operations. Then, we formally state our problem.

## 2.1 Definitions

A matrix is a rectangular array of *scalars*, that is, entries from a ring $R$; the entries within a matrix are called its elements. The followings are examples of matrices, first over the integer ring $\mathbb{Z}$

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix},$$

then over the polynomial ring $\mathbb{Z}[x]$,

$$\begin{bmatrix} 1+x & 1+x^2 \\ 1+x^3 & 1+x+x^2 \end{bmatrix}.$$

In these examples, the ring $R$ was commutative; this does not have to be the case. The next example is a matrix over a differential operator ring, where $\partial$ is the differentiation operation over $x$ i.e. $\frac{d}{dx}$,

$$\begin{bmatrix} (1+x)+(1+x^2)\partial & 1+(1+x^2)\partial^2 \\ (1+x^3)\partial & 1+(1+x+x^2)\partial \end{bmatrix};$$

the final one has entries in a linear recurrence operator ring, where $E$ is a shift operator.

$$\begin{bmatrix} (1+x)+(1+x^2)E & 1+(1+x^2)E^2 \\ (1+x^3)E^3 & 1+(1+x+x^2)E \end{bmatrix}.$$

We will go back to these examples, with proper definitions, in Chapter 8. Let us also mention the following extra conventions and notation:

1. Matrices are written in upper case.

2. The element in the $i$th row and $j$th column of a matrix $A$ shall be denoted by $a_{i,j}$. Thus if $A$ is a matrix with $m$ rows and $n$ columns, then

$$A = \begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & & \vdots \\ a_{m,1} & \cdots & a_{m,n} \end{bmatrix}.$$

3. The matrix $A$ with $m$ rows and $n$ columns may be written in abbreviated form as $A = [a_{i,j}]_{m\times n}$ or $A = [a_{i,j}]$.

4. The submatrices of a matrix $A$ will be written using notation such as $A^{i,j}$ (so as to allow the use of subscripts such as $a^{i,j}_{k,\ell}$ to denote the entries of $A^{i,j}$).

5. $\mathcal{M}_{m,n}(R)$ will denote the set of matrices of size $m \times n$ with entries in a ring $R$.

6. A product of size $\langle m, n, p\rangle$ is the product of a matrix of size $m \times n$ by a matrix of size $n \times p$.

Sum and differences of matrices are easy to define. Given two matrices $A = [a_{i,j}]_{m\times n}$ and $B = [b_{i,j}]_{m\times n}$, the sum $A + B$ is the $m \times n$ matrix $C = [c_{i,j}]$ where $c_{i,j} = a_{i,j} + b_{i,j}$, for each pair of subscripts $(i, j)$. The difference $D = A - B$ is the $m \times n$ matrix $D = [d_{i,j}]$ where $d_{i,j} = a_{i,j} - b_{i,j}$, for each pair of subscripts $(i, j)$.

If $A = [a_{i,j}]_{m\times n}$ and $B = [b_{i,j}]_{n\times p}$, then the product $AB$ is the $m \times p$ matrix $C = [c_{i,j}]_{m\times p}$, where

$$c_{i,j} = \sum_{k=1}^{n} a_{i,k}b_{k,j} = a_{i,1}b_{1,j} + a_{i,2}b_{2,j} + \cdots + a_{i,n}b_{n,j}.$$

We can define the product as above provided that the number of columns of $A$ is equal to the number of rows of $B$. Remark that matrix multiplication is not commutative, except for $1 \times 1$ matrices.

## 2.2 Computational model

To state our goal, we first need to clarify what matrix multiplication algorithms we allow.

### 2.2.1 The non-commutative case

We consider matrices with entries in a ring $R$, which we do not assume to be commutative. Then, to multiply two matrices $A$ and $B$, our algorithms for matrix multiplication are required to follow the following scheme:

1. compute linear combinations $\alpha_1, \ldots, \alpha_L$ (resp. $\beta_1, \ldots, \beta_L$) of the entries of the input matrix $A$ (resp. $B$);
2. multiply pairwise the values thus obtained, to obtain $\gamma_1 = \alpha_1\beta_1, \ldots, \gamma_L = \alpha_L\beta_L$
3. obtain the entries of $C = AB$ as linear combinations of $\gamma_1, \ldots, \gamma_L$.

Note that indeed, Strassen's algorithm, but also the naive algorithm, follow this approach. To be completely formal, one should also specify what computational model is used to perform the linear combinations: one can for instance use linear graphs, as in [6, Ch. 13].

An alternative viewpoint will be useful. A *matrix multiplication pattern* over $R$, of length $L$ and format $(m, n, p)$, is the data of three sequences of matrices

1. $(U_k)_{k\leq L}$, with $U_k$ in $\mathcal{M}_{m,n}(R)$
2. $(V_k)_{k\leq L}$, with $V_k$ in $\mathcal{M}_{n,p}(R)$
3. $(W_k)_{k\leq L}$, with $W_k$ in $\mathcal{M}_{m,p}(R)$,

such that the following holds. Let $\mathbf{A} = [\mathbf{a}_{i,j}]_{m\times n}$ and $\mathbf{B} = [\mathbf{b}_{i,j}]_{n\times p}$ be matrices whose entries are variables (hence the bold face, to distinguish them from scalars); for $k = 1, \ldots, L$, define

- $\alpha_k = \sum_{1\leq m}\sum_{j\leq n} u_{k,i,j}\mathbf{a}_{i,j}$
- $\beta_k = \sum_{1\leq n}\sum_{j\leq p} v_{k,i,j}\mathbf{b}_{i,j}$
- $\gamma_k = \alpha_k\beta_k$,

and define finally $\mathbf{c}_{i,j} = \sum_{k \leq L} w_{k,i,j}\gamma_k$ for $i \leq m$ and $j \leq p$. Then, we ask that

$$\mathbf{c}_{i,j} = \sum_{k=1}^{n} \mathbf{a}_{i,k}\mathbf{b}_{k,j}$$

for all $i, j$. If this is the case, we can indeed use $(U_k)_{k \leq L}$, $(V_k)_{k \leq L}$ and $(W_k)_{k \leq L}$ to multiply matrices with coefficients in $R$.

For example the pattern describing Strassen's algorithm has length 7 and format $(2, 2, 2)$, and holds over any ring $R$; it is given by

$$U_1 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad V_1 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$
$$U_2 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \quad V_2 = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1 & 0 \\ 0 & 0 \end{bmatrix}$$
$$U_3 = \begin{bmatrix} 1 & 1 \\ -1 & -1 \end{bmatrix}, \quad V_3 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, \quad W_3 = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$
$$U_4 = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}, \quad V_4 = \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad W_4 = \begin{bmatrix} 0 & 0 \\ -1 & 0 \end{bmatrix}$$
$$U_5 = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}, \quad V_5 = \begin{bmatrix} -1 & 1 \\ 0 & 0 \end{bmatrix}, \quad W_5 = \begin{bmatrix} 0 & 1 \\ 0 & 1 \end{bmatrix}$$
$$U_6 = \begin{bmatrix} -1 & 0 \\ 1 & 1 \end{bmatrix}, \quad V_6 = \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix}, \quad W_6 = \begin{bmatrix} 0 & 1 \\ 1 & 1 \end{bmatrix}$$
$$U_7 = \begin{bmatrix} 1 & 0 \\ -1 & 0 \end{bmatrix}, \quad V_7 = \begin{bmatrix} 0 & -1 \\ 0 & 1 \end{bmatrix}, \quad W_7 = \begin{bmatrix} 0 & 0 \\ 1 & 1 \end{bmatrix}$$

Indeed, from the first three matrices we deduce that we should compute $\gamma_1 = a_{1,1}b_{1,1}$, and that $\gamma_1$ will be used in the result matrix for entries $c_{1,1}, \ldots, c_{2,2}$, with coefficients 1 every time.

Giving a pattern is almost enough to specify a matrix multiplication algorithm. It is not quite sufficient, as the pattern does not indicate how to perform the linear combinations (e.g., by sharing common subexpressions). This is however a minor issue; in our implementations, we use a naive algorithm to perform the linear combinations.

### 2.2.2 Block matrix multiplication

It is well-known that we can use patterns in a recursive way through block matrix multiplication. This is done by subdividing (partitioning) the elements of a matrix into groups of elements, each of which itself a matrix; each such group is called a *submatrix* of the original matrix, or a *block*.

We already mentioned in the introduction that we can use Strassen's pattern to multiply matrices of size that are powers of 2. For instance, in size 4, the inputs $A, B$ are split into blocks of size 2:

$$A = \begin{bmatrix} A^{1,1} & A^{1,2} \\ A^{2,1} & A^{2,2} \end{bmatrix}, \quad B = \begin{bmatrix} B^{1,1} & B^{1,2} \\ B^{2,1} & B^{2,2} \end{bmatrix};$$

the product $C = AB$ is split similarly:

$$C = \begin{bmatrix} C^{1,1} & C^{1,2} \\ C^{2,1} & C^{2,2} \end{bmatrix}.$$

Then, Strassen's pattern is used to obtain all $C^{i,j}$ by means of 7 multiplications of blocks of size 2. Each of these multiplications is itself obtained used Strassen's pattern, for a total of 49 multiplications. Continuing, we obtain multiplication algorithms for sizes of the form $2^k$.

Similar subdivisions can be employed for any given pattern. Formally, suppose we want to apply a pattern $(U_k, V_k, W_k)_{k \leq L}$ of format $(a, b, c)$ and length $L$ to perform a product $C = AB$ of size $\langle m, n, p \rangle$. In the simplest case, we assume that

- $a$ divides $m$
- $b$ divides $n$
- $c$ divides $p$

Then, the subdivisions of $A$, $B$ and $C$ are straightforward: we subdivide $A$ into blocks $A^{i,j}$ of size $m/a \times n/b$, $B$ into blocks $B^{i,j}$ of size $n/b \times p/c$ and $C$ into blocks $C^{i,j}$ of size $m/a \times p/c$. Then we will have

$$A = \begin{bmatrix} A^{1,1} & \cdots & A^{1,b} \\ \vdots & & \vdots \\ A^{a,1} & \cdots & A^{a,b} \end{bmatrix}, \quad B = \begin{bmatrix} B_{1,1} & \cdots & B_{1,c} \\ \vdots & & \vdots \\ B_{b,1} & \cdots & B_{b,c} \end{bmatrix},$$

with result matrix

$$C = \begin{bmatrix} C_{1,1} & \cdots & C_{1,c} \\ \vdots & & \vdots \\ C_{a,1} & \cdots & C_{a,c} \end{bmatrix}.$$

For $k = 1, \ldots, L$, we compute

- $\alpha_k = \sum_{i \leq m} \sum_{j \leq n} u_{k,i,j} A^{i,j}$
- $\beta_k = \sum_{i \leq n} \sum_{j \leq p} v_{k,i,j} B^{i,j}$
- $\gamma_k = \alpha_k \beta_k$,

and we obtain $C^{i,j} = \sum_{k \leq L} w_{k,i,j} \gamma_k$.

This technique requires adaptation when the matrix size is not a power of the pattern size: typically, one can either *pad* the input matrices using extra rows or columns of zeros, or *peel* it from a few rows or columns. This is harmless as far as asymptotic estimates are concerned, since it only induces a constant factor overhead, but can be significant in practice. We will return to this question in Chapter 4.

Using a pattern of length $L$ and format $(a, a, a)$ gives an algorithm of complexity $O(n^{\log_a(L)})$ for $n \times n$ multiplication. It is actually also possible to use a non-square pattern of length $L$ and format $(a, b, c)$; using techniques mentioned in Chapter 3, this results in a complexity $O(n^{3 \log_{abc}(L)})$ for $n \times n$ multiplication.

### 2.2.3 Matrices with commutative entries

Finally, we mention the special case (though quite useful in practice) of matrices defined over commutative rings. Commutativity means that we can use the relations $a_{i,j} b_{k,\ell} = b_{k,\ell} a_{i,j}$ to design our algorithms. Thus, a *commutative algorithm* will follow the following scheme:

- compute linear combinations $\alpha_1, \ldots, \alpha_L$ and $\beta_1, \ldots, \beta_L$ of the entries of the input matrices $A$ and $B$: here, both $\alpha_i$'s and $\beta_i$'s can involve entries of $A$ and $B$;
- multiply pairwise the values thus obtained, to get $\gamma_1 = \alpha_1 \beta_1, \ldots, \gamma_L = \alpha_L \beta_L$
- obtain the entries of $C = AB$ as linear combinations of $\gamma_1, \ldots, \gamma_L$.

Commutative algorithms play in the literature less important a role than the non-commutative ones, since the commutativity assumption prevents one from using them recursively (the product of block matrices is not commutative).

Note that it is possible to describe a commutative algorithm using a similar data as non-commutative ones, that is, sequences $(U_k)_{k\leq L}$, $(V_k)_{k\leq L}$ and $(W_k)_{k\leq L}$, where now each $U_k$ and $V_k$ involves two matrices (one that gives the coefficients applied to the first input $A$, and that other one which gives the coefficients applied to the second input $B$). The following is an example of a pattern for a commutative algorithm, due to Waksman [29], whose extension to larger matrices is presented in more detail in Chapter 6.

$$
\begin{aligned}
U_1 &= \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}0 & 0\\ 1 & 0\end{bmatrix}, & V_1 &= \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}, & W_1 &= \begin{bmatrix}\frac{1}{2} & 0\\ 0 & \frac{1}{2}\end{bmatrix}\\
U_2 &= \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}0 & 0\\ 0 & 1\end{bmatrix}, & V_2 &= \begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}, & W_2 &= \begin{bmatrix}0 & \frac{1}{2}\\ 0 & -\frac{1}{2}\end{bmatrix}\\
U_3 &= \begin{bmatrix}0 & 0\\ 1 & 0\end{bmatrix}\begin{bmatrix}0 & 0\\ 1 & 0\end{bmatrix}, & V_3 &= \begin{bmatrix}0 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}, & W_3 &= \begin{bmatrix}0 & 0\\ \frac{1}{2} & -\frac{1}{2}\end{bmatrix}\\
U_4 &= \begin{bmatrix}0 & 0\\ 1 & 0\end{bmatrix}\begin{bmatrix}0 & 0\\ 0 & 1\end{bmatrix}, & V_4 &= \begin{bmatrix}0 & 0\\ 0 & 1\end{bmatrix}\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}, & W_4 &= \begin{bmatrix}0 & 0\\ 0 & 1\end{bmatrix}\\
U_5 &= \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}0 & 0\\ -1 & 0\end{bmatrix}, & V_5 &= \begin{bmatrix}0 & -1\\ 0 & 0\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}, & W_5 &= \begin{bmatrix}\frac{1}{2} & 0\\ 0 & -\frac{1}{2}\end{bmatrix}\\
U_6 &= \begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}\begin{bmatrix}0 & 0\\ 0 & -1\end{bmatrix}, & V_6 &= \begin{bmatrix}0 & -1\\ 0 & 0\end{bmatrix}\begin{bmatrix}0 & 1\\ 0 & 0\end{bmatrix}, & W_6 &= \begin{bmatrix}0 & \frac{1}{2}\\ 0 & \frac{1}{2}\end{bmatrix}\\
U_7 &= \begin{bmatrix}0 & 0\\ 1 & 0\end{bmatrix}\begin{bmatrix}0 & 0\\ -1 & 0\end{bmatrix}, & V_7 &= \begin{bmatrix}0 & 0\\ 0 & -1\end{bmatrix}\begin{bmatrix}1 & 0\\ 0 & 0\end{bmatrix}, & W_7 &= \begin{bmatrix}0 & 0\\ \frac{1}{2} & \frac{1}{2}\end{bmatrix}.
\end{aligned}
$$

# Chapter 3

# Previous work

## 3.1 Introduction

Since the 1960's, many algorithms were developed to multiply matrices fast. As was explained in the introduction, many previous results on matrix multiplications were concerned with asymptotic estimates only, yielding very complex algorithms, that are impractical for any matrix of reasonable size. On the other hand, there also exist a handful of simple and practically useful techniques; this chapter describes such classical algorithms.

## 3.2 Non-commutative algorithms

We start with the most well-known family of algorithms, the non-commutative ones, where the base ring is not assumed to be commutative.

### 3.2.1 Strassen's algorithm and Winograd's variant

The version of Strassen's algorithm given in the introduction is actually a modification due to Winograd. On input $2 \times 2$ matrices $A$ and $B$, the 7 product terms of Strassen's original construction are

- $\gamma_1 = a_{2,2}(b_{2,1} - b_{1,1})$
- $\gamma_2 = a_{1,1}(b_{1,2} - b_{2,2})$
- $\gamma_3 = (a_{2,1} + a_{2,2})b_{1,1}$
- $\gamma_4 = (a_{1,1} + a_{1,2})b_{2,2}$

- $\gamma_5 = (a_{2,1} - a_{1,1})(b_{1,1} + b_{1,2})$
- $\gamma_6 = (a_{1,2} - a_{2,2})(b_{2,1} + b_{2,2})$
- $\gamma_7 = (a_{1,1} + a_{2,2})(b_{1,1} + b_{2,2})$,

and the result is

- $c_{1,1} = \gamma_1 + \gamma_6 + \gamma_7 - \gamma_4$
- $c_{1,2} = \gamma_2 + \gamma_4$
- $c_{2,1} = \gamma_1 + \gamma_3$
- $c_{2,2} = \gamma_2 - \gamma_3 + \gamma_5 + \gamma_7$.

Sharing common expressions, one can perform the above operations using 18 additions / subtractions. While the number of additions is not our main concern, it motivated the design by Winograd [31] of his variant (already presented in the introduction), where sharing common expressions allows to reduce the addition / subtraction count to 15. The 7 products are now

- $\gamma_1 = a_{1,1}b_{1,1}$
- $\gamma_2 = a_{1,2}b_{2,1}$
- $\gamma_3 = (a_{1,1} + a_{1,2} - a_{2,1} - a_{2,2})b_{2,2}$
- $\gamma_4 = a_{2,2}(b_{1,1} - b_{1,2} - b_{2,1} + b_{2,2})$
- $\gamma_5 = (a_{2,1} + a_{2,2})(-b_{1,1} + b_{1,2})$
- $\gamma_6 = (a_{2,1} + a_{2,2} - a_{1,1})(b_{2,2} - b_{1,2} + b_{1,1})$
- $\gamma_7 = (a_{1,1} - a_{2,1})(-b_{1,2} + b_{2,2})$,

and the result is

- $c_{1,1} = \gamma_1 + \gamma_2$
- $c_{1,2} = \gamma_1 + \gamma_3 + \gamma_5 + \gamma_6$
- $c_{2,1} = \gamma_1 - \gamma_4 + \gamma_6 + \gamma_7$
- $c_{2,2} = \gamma_1 + \gamma_5 + \gamma_6 + \gamma_7$.

For example, given the input matrices

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

we obtain

- $\gamma_1 = 1 \times 1 = 1$
- $\gamma_2 = 2 \times 3 = 6$
- $\gamma_3 = (2 - 3 - 4 + 1) \times 4 = -16$
- $\gamma_4 = 4 \times (1 - 2 - 3 + 4) = 0$
- $\gamma_5 = (3 + 4) \times (-1 + 2) = 7$
- $\gamma_6 = (3 + 4 - 1) \times (4 - 2 + 1) = 18$
- $\gamma_7 = (1 - 3) \times (-2 + 4) = -4,$

and the result is

- $c_{1,1} = 1 + 6 = 7$
- $c_{1,2} = 1 - 16 + 7 + 18 = 10$
- $c_{2,1} = 1 - 0 + 18 - 4 = 15$
- $c_{2,2} = 1 + 7 + 18 - 4 = 22.$

### 3.2.2 Laderman's Algorithm

In 1975, Laderman [16] developed a non-commutative algorithm for multiplying $3 \times 3$ matrices using 23 multiplications. This is algorithm is still the best known one for the $3 \times 3$ case, even though its exponent is not as good as Strassen's. Given two $3 \times 3$ matrices $A = [a_{i,j}]$ and $B = [b_{i,j}]$, the 23 product terms are

- $\gamma_1 = (a_{1,1} + a_{1,2} + a_{1,3} - a_{2,1} - a_{2,2} - a_{3,2} - a_{3,3})b_{2,2}$
- $\gamma_2 = (a_{1,1} - a_{2,1})(-b_{1,2} + b_{2,2})$
- $\gamma_3 = a_{2,2}(-b_{1,1} + b_{1,2} + b_{2,1} - b_{2,2} - b_{2,3} - b_{3,1} + b_{3,3})$
- $\gamma_4 = (-a_{1,1} + a_{2,1} + a_{2,2})(b_{1,1} - b_{1,2} + b_{2,2})$

- $\gamma_5 = (a_{2,1} + a_{2,2})(-b_{1,1} + b_{1,2})$
- $\gamma_6 = a_{1,1}b_{1,1}$
- $\gamma_7 = (-a_{1,1} + a_{3,1} + a_{3,2})(b_{1,1} - b_{1,3} + b_{2,3})$
- $\gamma_8 = (-a_{1,1} + a_{3,1})(b_{1,3} - b_{2,3})$
- $\gamma_9 = (a_{3,1} + a_{3,2})(-b_{1,1} + b_{1,3})$
- $\gamma_{10} = (a_{1,1} + a_{1,2} + a_{1,3} - a_{2,2} - a_{2,3} - a_{3,1} - a_{3,3})b_{2,3}$
- $\gamma_{11} = a_{3,2}(-b_{1,1} + b_{1,3} + b_{2,1} - b_{2,2} - b_{2,3} - b_{3,1} + b_{3,2})$
- $\gamma_{12} = (-a_{1,3} + a_{3,2} + a_{3,3})(b_{2,2} + b_{3,1} - b_{3,2})$
- $\gamma_{13} = (a_{1,3} - a_{3,3})(b_{2,2} - b_{3,2})$
- $\gamma_{14} = a_{1,3}b_{3,1}$
- $\gamma_{15} = (a_{3,2} + a_{3,3})(-b_{3,1} + b_{3,2})$
- $\gamma_{16} = (-a_{1,3} + a_{2,2} + a_{2,3})(b_{2,3} + b_{3,1} - b_{3,3})$
- $\gamma_{17} = (a_{1,3} - a_{2,3})(b_{2,3} - b_{3,3})$
- $\gamma_{18} = (a_{2,2} + a_{2,3})(-b_{3,1} + b_{3,3})$
- $\gamma_{19} = a_{1,2}b_{2,1}$
- $\gamma_{20} = a_{2,3}b_{3,2}$
- $\gamma_{21} = a_{2,1}b_{1,3}$
- $\gamma_{22} = a_{3,1}b_{1,2}$
- $\gamma_{23} = a_{3,3}b_{3,3}$,

and the result is

- $c_{1,2} = \gamma_1 + \gamma_4 + \gamma_5 + \gamma_6 + \gamma_{12} + \gamma_{14} + \gamma_{15}$
- $c_{1,3} = \gamma_6 + \gamma_7 + \gamma_9 + \gamma_{10} + \gamma_{14} + \gamma_{16} + \gamma_{18}$
- $c_{2,1} = \gamma_2 + \gamma_3 + \gamma_4 + \gamma_6 + \gamma_{14} + \gamma_{16} + \gamma_{17}$
- $c_{2,2} = \gamma_2 - \gamma_4 + \gamma_5 + \gamma_6 + \gamma_{20}$

- $c_{2,3} = \gamma_{14} + \gamma_{16} + \gamma_{17} + \gamma_{18} + \gamma_{21}$
- $c_{3,1} = \gamma_6 + \gamma_7 + \gamma_8 + \gamma_{11} + \gamma_{12} + \gamma_{13} + \gamma_{14}$
- $c_{3,2} = \gamma_{12} + \gamma_{13} + \gamma_{14} + \gamma_{15} + \gamma_{22}$
- $c_{3,3} = \gamma_6 + \gamma_7 + \gamma_8 + \gamma_9 + \gamma_{23}$

### 3.2.3 Hopcroft and Kerr's algorithm

In 1970, Hopcroft and Kerr [13] developed an algorithm to multiply a $p \times 2$ matrix by a $2 \times n$ matrix in $\lceil (3pn + \max(n,p))/2 \rceil$ multiplications, without the use of commutativity. We give here the operations for the case of $p = n = 3$, with 15 multiplications. Given two matrices $A = [a_{i,j}]_{3\times 2}$ and $B = [b_{i,j}]_{2\times 3}$, the 15 products are

- $\gamma_1 = (a_{1,1} - a_{1,2})b_{1,1}$
- $\gamma_2 = a_{1,2}(b_{1,1} + b_{2,1})$
- $\gamma_3 = a_{2,1}b_{1,2}$
- $\gamma_4 = a_{2,2}b_{2,2}$
- $\gamma_5 = a_{3,1}(b_{1,3} + b_{2,3})$
- $\gamma_6 = (-a_{3,1} + a_{3,2})b_{2,3}$
- $\gamma_7 = (a_{1,1} + a_{2,1})(b_{1,1} + b_{1,2} + b_{2,1} + b_{2,2})$
- $\gamma_8 = (a_{1,1} - a_{1,2} + a_{2,1})(b_{1,1} + b_{2,1} + b_{2,2})$
- $\gamma_9 = (a_{1,1} - a_{1,2} + a_{2,1} - a_{2,2})(b_{2,1} + b_{2,2})$
- $\gamma_{10} = (a_{2,2} + a_{3,2})(b_{1,2} + b_{1,3} + b_{2,2} + b_{2,3})$
- $\gamma_{11} = (a_{2,2} - a_{3,1} + a_{3,2})(b_{1,2} + b_{1,3} + b_{2,3})$
- $\gamma_{12} = (-a_{2,1} + a_{2,2} - a_{3,1} + a_{3,2})(b_{1,2} + b_{1,3})$
- $\gamma_{13} = (a_{1,2} + a_{3,1})(b_{1,1} - b_{2,3})$
- $\gamma_{14} = (-a_{1,2} - a_{3,2})(b_{2,1} + b_{2,3})$
- $\gamma_{15} = (a_{1,1} + a_{3,1})(b_{1,1} + b_{1,3})$,

and the result is

- $c_{1,1} = \gamma_1 + \gamma_2$
- $c_{1,2} = -\gamma_2 - \gamma_3 + \gamma_7 - \gamma_8$
- $c_{1,3} = -\gamma_1 - \gamma_5 - \gamma_{13} + \gamma_{15}$
- $c_{2,1} = -\gamma_1 - \gamma_4 + \gamma_8 - \gamma_9$
- $c_{2,2} = \gamma_3 + \gamma_4$
- $c_{2,3} = -\gamma_3 - \gamma_6 + \gamma_{11} - \gamma_{12}$
- $c_{3,1} = -\gamma_2 - \gamma_6 + \gamma_{13} - \gamma_{14}$
- $c_{3,2} = -\gamma_4 - \gamma_5 + \gamma_{10} - \gamma_{11}$
- $c_{3,3} = \gamma_5 + \gamma_6$.

### 3.2.4 Duality

Hopcroft and Musinski [12] defined the *dual* of a multiplication algorithm, and applied this idea to matrix multiplication. An $\langle n, m, p\rangle$ matrix product is shown to be the dual of an $\langle n, p, m\rangle$ product, or of an $\langle m, n, p\rangle$ product. The dual of a set of expressions representing the multiplication of two matrices is a set of expressions of the same length. Thus, given an $\langle m, n, p\rangle$ problem, we can replace it by any problem $\langle u, v, w\rangle$, where $\langle u, v, w\rangle$ is one of the six permutations of $\langle m, n, p\rangle$, without changing the number of multiplications.

As an aside, note that using this, one can justify the remark made in the previous chapter, that rectangular shaped patterns can be used to multiply square matrices.

As an illustration, starting from the previous algorithm for $3 \times 2$ by $2 \times 3$ matrix multiplication, and using their construction, Hopcroft and Musinski obtained the following algorithm for $3 \times 3$ by $3 \times 2$ products. Given two matrices $A = [a_{i,j}]_{3\times 3}$ and $B = [b_{i,j}]_{3\times 2}$, the 15 products are

- $\gamma_1 = (a_{1,1} - a_{1,3} - a_{2,1})b_{1,1}$
- $\gamma_2 = (a_{1,1} - a_{1,2} - a_{3,1})(b_{1,1} + b_{1,2})$
- $\gamma_3 = (-a_{1,2} + a_{2,2} - a_{2,3})b_{2,1}$
- $\gamma_4 = (-a_{2,1} + a_{2,2} - a_{3,2})b_{2,2}$

- $\gamma_5 = (-a_{1,3} - a_{3,2} + a_{3,3})(b_{3,1} + b_{3,2})$
- $\gamma_6 = (-a_{2,3} - a_{3,1} + a_{3,3})b_{3,2}$
- $\gamma_7 = a_{1,2}(b_{1,1} + b_{1,2} + b_{2,1} + b_{2,2})$
- $\gamma_8 = (-a_{1,2} + a_{2,1})(b_{1,1} + b_{1,2} + b_{2,2})$
- $\gamma_9 = -a_{2,1}(b_{1,2} + b_{2,2})$
- $\gamma_{10} = a_{3,2}(b_{2,1} + b_{2,2} + b_{3,1} + b_{3,2})$
- $\gamma_{11} = (a_{2,3} - a_{3,2})(b_{2,1} + b_{3,1} + b_{3,2})$
- $\gamma_{12} = -a_{2,3}(b_{2,1} + b_{3,1})$
- $\gamma_{13} = (-a_{1,3} + a_{3,1})(b_{1,1} - b_{3,2})$
- $\gamma_{14} = -a_{3,1}(b_{1,2} + b_{3,2})$
- $\gamma_{15} = a_{1,3}(b_{1,1} + b_{3,1})$,

and the result is

- $c_{1,1} = \gamma_1 + \gamma_7 + \gamma_8 + \gamma_9 + \gamma_{15}$
- $c_{1,2} = -\gamma_1 + \gamma_2 - \gamma_8 - \gamma_9 + \gamma_{13} - \gamma_{14}$
- $c_{2,1} = \gamma_3 + \gamma_7 + \gamma_8 + \gamma_9 - \gamma_{12}$
- $c_{2,2} = \gamma_4 - \gamma_9 + \gamma_{10} + \gamma_{11} + \gamma_{12}$
- $c_{3,1} = \gamma_5 - \gamma_6 - \gamma_{11} - \gamma_{12} + \gamma_{13} + \gamma_{15}$
- $c_{3,2} = \gamma_6 + \gamma_{10} + \gamma_{11} + \gamma_{12} - \gamma_{14}$.

Using the same approach, we devised an algorithm for $2 \times 3$ by $3 \times 3$ products. Given two matrices $A = [a_{i,j}]_{2\times 3}$ and $B = [b_{i,j}]_{3\times 3}$, the 15 products are

- $\gamma_1 = (a_{1,1} - a_{2,1})(b_{1,1} - b_{1,3} - b_{2,1})$
- $\gamma_2 = a_{2,1}(b_{1,1} - b_{3,1} - b_{1,2})$
- $\gamma_3 = a_{1,2}(b_{2,2} - b_{2,3} - b_{1,2})$
- $\gamma_4 = a_{2,2}(b_{2,2} - b_{2,1} - b_{3,2})$

- $\gamma_5 = a_{1,3}(b_{3,3} - b_{3,2} - b_{1,3})$
- $\gamma_6 = (a_{2,3} - a_{1,3})(b_{3,3} - b_{3,1} - b_{2,3})$
- $\gamma_7 = (a_{1,1} + a_{1,2})b_{1,2}$
- $\gamma_8 = (a_{1,1} + a_{1,2} - a_{2,1})(b_{2,1} - b_{1,2})$
- $\gamma_9 = (a_{1,1} + a_{1,2} - a_{2,1} - a_{2,2})(-b_{2,1})$
- $\gamma_{10} = (a_{2,2} + a_{2,3})b_{3,2}$
- $\gamma_{11} = (a_{2,2} + a_{2,3} - a_{1,3})(b_{2,3} - b_{3,2})$
- $\gamma_{12} = (a_{2,2} - a_{1,2} + a_{2,3} - a_{1,3})(-b_{2,3})$
- $\gamma_{13} = (a_{2,1} + a_{1,3})(b_{3,1} - b_{1,3})$
- $\gamma_{14} = (-a_{2,1} - a_{2,3})(-b_{3,1})$
- $\gamma_{15} = (a_{1,1} + a_{1,3})b_{1,3}$,

and the result is

- $c_{1,1} = \gamma_1 + \gamma_2 + \gamma_7 + \gamma_8 + \gamma_{13} + \gamma_{15}$
- $c_{1,2} = \gamma_3 + \gamma_7 + \gamma_{10} + \gamma_{11} + \gamma_{12}$
- $c_{1,3} = \gamma_5 + \gamma_{10} + \gamma_{11} + \gamma_{12} + \gamma_{15}$
- $c_{2,1} = \gamma_2 + \gamma_7 + \gamma_8 + \gamma_9 + \gamma_{14}$
- $c_{2,2} = \gamma_4 + \gamma_7 + \gamma_8 + \gamma_9 + \gamma_{10}$
- $c_{2,3} = \gamma_5 + \gamma_6 + \gamma_{10} + \gamma_{11} + \gamma_{14} - \gamma_{13}$.

### 3.2.5 Makarov's algorithm

Makarov [19] proposed a non-commutative algorithm for multiplying two square $5 \times 5$ matrices using 100 multiplications, which gives an exponent $\omega \leq 2.862$. This is the best known algorithm for the $5 \times 5$ case. The original problem is decomposed into the following five subproblems

- a $3 \times 3$ product, in 23 multiplications by Laderman's algorithm;

- a $2 \times 3$ by $3 \times 3$ product, a $3 \times 3$ by $3 \times 2$ product, and a $3 \times 2$ by $2 \times 3$ product; all of them in 15 multiplication by Hopcroft and Musinski's algorithm;
- a $2 \times 2$ product, in 7 multiplications by Strassen's algorithm;
- a $2 \times 2$ by $2 \times 3$ product, in 11 multiplication by Strassen's algorithm;
- a $3 \times 3$ by $3 \times 2$ product, where the first matrix has a 0 as an entry; this is done in 14 multiplication by modifying Hopcroft and Musinski's algorithm.

In total, it costs $23 + 45 + 7 + 11 + 14 = 100$ multiplications.

### 3.2.6 Two disjoint matrix multiplications

Pan [21] proposed a practical algorithm for square matrix multiplication based on a formula to perform *two* matrix products simultaneously; Kaporin [14] later gave a slightly better algorithms in terms of additions. Suppose we want to compute

$$Z = XY, \quad W = UV,$$

where the matrices $Z, X, Y$ and $W, U, V$ are rectangular of sizes $m \times p$, $m \times n$, $n \times p$, and $n \times m$, $n \times p$ and $p \times m$ respectively. The naive algorithm has $2mnp$ products; Pan's improved formula is as follows:

$$z_{i,j} = \sum_{k=1}^{n} (x_{i,k} + u_{k,j})(y_{k,j} + v_{j,i}) - \sum_{k=1}^{n} u_{k,j} y_{k,j} - (\sum_{k=1}^{n} x_{i,k} + \sum_{k=1}^{n} u_{k,j}) v_{j,i}$$

$$i = 1, \ldots, m, \quad j = 1, \ldots, p$$

$$w_{k,i} = \sum_{j=1}^{p} (x_{i,k} + u_{k,j})(y_{k,j} + v_{j,i}) - \sum_{j=1}^{p} u_{k,j} y_{k,j} - (\sum_{j=1}^{p} y_{k,j} + \sum_{j=1}^{p} v_{j,i}) x_{i,k}$$

$$k = 1, \ldots, n, \quad i = 1, \ldots, m.$$

Due to the shared expressions in these formulas, the cost is reduced to $mnp + mn + np + pm$ multiplications. Remark that, up to neglecting the lower-order terms $mn + np + pm$, this is about twice better than the naive algorithm.

From this, it is possible to deduce a recursive algorithm for a single matrix multiplication. We do not need it, as it does not give better multiplication counts than other approaches. However, the disjoint product formula above will be used to generate our tables.

### 3.2.7 Three disjoint matrix multiplications

Pan [21] also proposed a practical algorithm for square matrix multiplication based on *three* simultaneous matrix products. As for the previous example, a formula is found for three simultaneous products, and an algorithm for a single product is deduced.

This approach, and some variants (that save some multiplications) were put to practice by Laderman, Pan and Sha [17] and Kaporin [15]. They showed how to perform a single $n \times n$ matrix multiplication in

$$\frac{n^3}{3} \; + \; \frac{1}{3}\min\{12n^2 + 17n, \frac{45}{4}n^2 + \frac{97}{2}n + 60\},$$

for $n$ even. This gives the smallest multiplication count previously known when $18 \leq \; 2n \leq \; 52$.

We do not give explicit formulas in this section, as they are quite complex. We devote Chapter 5 to this approach, and will show in particular how to improve on the bounds given above.

## 3.3 Commutative algorithms

For reasons explained in the previous chapter, fewer commutative algorithms are known; two are presented here. In all this section, we assume that our matrices are defined over commutative rings.

### 3.3.1 Makarov's algorithm

In [18], Makarov gives an algorithm using 22 multiplications for the product of $3 \times 3$ matrices (which better by 1 than Laderman's in the non-commutative case). Given two $3 \times 3$ matrices $A = [a_{i,j}]$ and $B = [b_{i,j}]$, the 22 product terms are

- $\gamma_1 = (b_{1,3} + a_{1,3} - a_{2,3})(a_{1,1} + b_{3,1} - b_{3,2} + b_{3,3})$
- $\gamma_2 = (b_{1,2} + a_{1,2} + a_{2,2})(a_{2,1} - b_{2,1} + b_{2,2} - b_{2,3})$
- $\gamma_3 = (b_{1,2} + a_{1,2} + a_{3,2})(a_{3,1} - b_{2,1} + b_{2,2} - b_{2,3})$
- $\gamma_4 = (b_{1,3} - a_{2,3} - a_{3,3})(a_{3,1} - b_{3,1} + b_{3,2} - b_{3,3})$
- $\gamma_5 = (b_{1,1} - a_{1,3} + a_{2,3})a_{1,1}$
- $\gamma_6 = (b_{1,1} + a_{1,2} + a_{2,2})a_{2,1}$

- $\gamma_7 = (b_{1,1} + a_{1,2} + a_{3,2} + a_{2,3} + a_{3,3})a_{3,1}$
- $\gamma_8 = b_{1,2}(a_{1,1} + b_{2,1} - b_{2,2} + b_{2,3})$
- $\gamma_9 = b_{1,3}(a_{2,1} + b_{3,1} - b_{3,2} + b_{3,3})$
- $\gamma_{10} = a_{1,2}b_{2,1}$
- $\gamma_{11} = a_{2,3}b_{3,1}$
- $\gamma_{12} = (a_{1,3} - a_{2,3})(a_{1,1} + b_{3,1})$
- $\gamma_{13} = (a_{1,2} + a_{2,2})(b_{2,1} - a_{2,1})$
- $\gamma_{14} = (a_{1,2} + b_{1,2})(b_{2,1} - b_{2,2} + b_{2,3})$
- $\gamma_{15} = a_{2,2}b_{2,3}$
- $\gamma_{16} = (b_{1,3} - a_{2,3})(b_{3,1} - b_{3,2} + b_{3,3})$
- $\gamma_{17} = a_{2,3}b_{3,2}$
- $\gamma_{18} = (a_{3,2} - a_{2,3} - a_{3,3})b_{3,2}$
- $\gamma_{19} = (a_{1,3} + a_{3,3} - a_{1,2} - a_{3,2})b_{3,2}$
- $\gamma_{20} = (a_{1,2} + a_{3,2})(b_{2,1} - a_{3,1} + b_{2,3} + b_{3,2})$
- $\gamma_{21} = (a_{2,3} + a_{3,3})(a_{3,1} + b_{2,3} - b_{3,1} + b_{3,2})$
- $\gamma_{22} = (a_{2,3} + a_{3,3} - a_{1,2} - a_{3,2})(b_{2,3} + b_{3,2})$.

The result is

- $c_{1,1} = \gamma_5 + \gamma_{10} + \gamma_{11} + \gamma_{12}$
- $c_{1,2} = \gamma_8 + \gamma_{10} - \gamma_{14} + \gamma_{17} - \gamma_{18} + \gamma_{19} - \gamma_{22}$
- $c_{1,3} = \gamma_1 - \gamma_{11} - \gamma_{12} - \gamma_{16} + \gamma_{17} - \gamma_{18} + \gamma_{19} - \gamma_{22}$
- $c_{2,1} = \gamma_6 - \gamma_{10} + \gamma_{11} + \gamma_{13}$
- $c_{2,2} = \gamma_2 - \gamma_{10} + \gamma_{13} + \gamma_{14} + \gamma_{15} + \gamma_{17}$
- $c_{2,3} = \gamma_9 - \gamma_{11} + \gamma_{15} - \gamma_{16} + \gamma_{17}$
- $c_{3,1} = \gamma_7 - \gamma_{10} - \gamma_{11} + \gamma_{20} - \gamma_{21} + \gamma_{22}$

- $c_{3,2} = \gamma_3 - \gamma_{10} + \gamma_{14} - \gamma_{17} + \gamma_{18} + \gamma_{20} + \gamma_{22}$
- $c_{3,3} = \gamma_4 + \gamma_{11} + \gamma_{16} - \gamma_{17} + \gamma_{18} + \gamma_{21}$.

### 3.3.2 Winograd and Waksman's algorithms

In [30], Winograd introduced an algorithm that allows one to reduce the number of multiplications for $n \times n$ products by almost half; Waksman [29] subsequently improved it. We do not give the details of this algorithm here, as we will present it in Chapter 6.

## 3.4 Previous tables

Before our work, one finds other attempts to record minimal number of multiplications for small matrices. We comment on two of them here; we will compare our results to the one obtained below in the following chapters.

### 3.4.1 Decomposition techniques by Probert and Fischer

Probert and Fischer [22] introduced several decomposition techniques for matrix multiplication. The basic idea, which we will follow as well, involves decomposing one matrix multiplication problem into smaller subproblems, and build a database incrementally. The decomposition rules they used are the following.

- *Permutation rule* (P). Given an $\langle m, n, p\rangle$ problem, replace it by any $\langle u, v, w\rangle$ problem, where $\langle u, v, w\rangle$ is one of the six permutations of $\langle m, n, p\rangle$. The cost is unchanged.
- *Additive Rule* (A). Given an $\langle m, n, p\rangle$ problem where $n = n_1 + n_2$, replace it by two subproblems of sizes $\langle m, n_1, p\rangle$ and $\langle m, n_2, p\rangle$. The cost is the sum of the costs of the two subproblems.
- *Multiplicative Rule* (M). Given an $\langle m, n, p\rangle$ problem where $m = i_1 i_2$, $n = j_1 j_2$ $p = k_1 k_2$, replace it by two subproblems of sizes $\langle i_1, j_1, k_1\rangle$ and $\langle i_2, j_2, k_2\rangle$ . The cost is the product of the costs of the two subproblems.
- *Zero Padding Rule* (Z). Given an $\langle m, n, p\rangle$ problem, replace it by an $\langle m, n+1, p\rangle$ problem. The cost is unchanged.

Using all these decomposition techniques they tabulated upper bounds for square case matrix multiplication problem up to dimension $40 \times 40 \times 40$. Here is an example of their derivations.

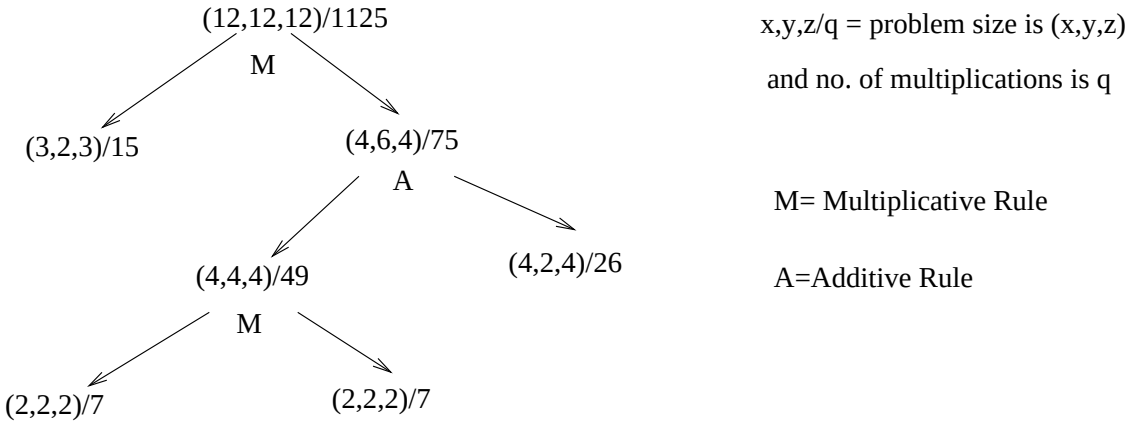



Figure 3.1: Decomposition for $\langle 12, 12, 12 \rangle$

Here, the problem $\langle 12, 12, 12 \rangle$ is divided as a product (using multiplicative rule) of two subproblems $\langle 3, 2, 3 \rangle$ and $\langle 4, 6, 4 \rangle$. Then, the problem $\langle 4, 6, 4 \rangle$ is divided as the sum of two subproblems $\langle 4, 4, 4 \rangle$ and $\langle 4, 2, 4 \rangle$. The problem $\langle 4, 4, 4 \rangle$ is solved using Strassen's algorithm and block product of matrices. The problem $\langle 4, 2, 4 \rangle$ is solved using Hopcroft and Kerr's technique. In this way applying divide and conquer approach the final result is found 1125.

As far as we can tell, their table was obtained by a manual search. Our approach will be able to find out such division automatically (and usually, to find better results).

### 3.4.2 Smith's table

In 2002, Smith [26] tabulated upper bounds for matrix multiplication for all possible rectangular dimensions up $\langle 11, 11, 11 \rangle$, and all square cases up to $\langle 28, 28, 28 \rangle$, which he obtained using a computer search using the patterns above. His table is quite informative and much larger than Probert and Fischer's. Our results are at least as good as his, and usually improve on them.

# Chapter 4

# An alternative to peeling and padding

## 4.1 Introduction

In this chapter, we suppose that we are given a pattern $(U_k, V_k, W_k)$ of length $L$ and format $(a, b, c)$. We have seen in Chapter 2 that if $(m, n, p)$ are such that $a, b, c$ respectively divide $m, n, p$, it is possible to use the pattern $(U_k, V_k, W_k)$ to perform multiplication in size $\langle m, n, p\rangle$ by performing block matrix multiplication, with blocks of size $(m/a, n/b, p/c)$.

When the divisibility condition does not hold, we have mentioned that padding with extra rows or columns of zeros, or peeling (removing) one row or column, provides a workaround. However, it is usually sub-optimal. Our goal in this chapter is to present a better alternative: we save multiplications, at the cost of performing an extra preparation depending on $(a, b, c)$ and $(m, n, p)$.

## 4.2 A worked example

To multiply two matrices of size $(3, 3)$, we may want to apply Strassen's pattern. The padding strategy mentioned before would consist in adding an extra row and column of zeros, to make the matrices $(4, 4)$, and proceed recursively. However, better can be done, by exploiting the presence of known zeros.

After padding the input matrices $A, B$ to give them size $(4, 4)$, we obtain

$$\tilde{A} = \begin{bmatrix} a_{1,1} & a_{1,2} & a_{1,3} & 0 \\ a_{2,1} & a_{2,2} & a_{2,3} & 0 \\ a_{3,1} & a_{3,2} & a_{3,3} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \tilde{B} = \begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} & 0 \\ b_{2,1} & b_{2,2} & b_{2,3} & 0 \\ b_{3,1} & b_{3,2} & b_{3,3} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The block decomposition is

$$\tilde{A}^{1,1} = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}, \quad \tilde{A}^{1,2} = \begin{bmatrix} a_{1,3} & 0 \\ a_{2,3} & 0 \end{bmatrix}, \quad \tilde{A}^{2,1} = \begin{bmatrix} a_{3,1} & a_{3,2} \\ 0 & 0 \end{bmatrix}, \quad \tilde{A}^{2,2} = \begin{bmatrix} a_{3,3} & 0 \\ 0 & 0 \end{bmatrix},$$

and

$$\tilde{B}^{1,1} = \begin{bmatrix} b_{1,1} & b_{1,2} \\ b_{2,1} & b_{2,2} \end{bmatrix}, \quad \tilde{B}^{1,2} = \begin{bmatrix} b_{1,3} & 0 \\ b_{2,3} & 0 \end{bmatrix}, \quad \tilde{B}^{2,1} = \begin{bmatrix} b_{3,1} & b_{3,2} \\ 0 & 0 \end{bmatrix}, \quad \tilde{B}^{2,2} = \begin{bmatrix} b_{3,3} & 0 \\ 0 & 0 \end{bmatrix}.$$

Recall that the product terms in the block version of Strassen's algorithm are, in this case

- $\gamma_1 = \tilde{A}^{2,2}(\tilde{B}^{2,1} - \tilde{B}^{1,1})$
- $\gamma_2 = \tilde{A}^{1,1}(\tilde{B}^{1,2} - \tilde{B}^{2,2})$
- $\gamma_3 = (\tilde{A}^{2,1} + \tilde{A}^{2,2})\tilde{B}^{1,1}$
- $\gamma_4 = (\tilde{A}^{1,1} + \tilde{A}^{1,2})\tilde{B}^{2,2}$
- $\gamma_5 = (\tilde{A}^{2,1} - \tilde{A}^{1,1})(\tilde{B}^{1,1} + \tilde{B}^{1,2})$
- $\gamma_6 = (\tilde{A}^{1,2} - \tilde{A}^{2,2})(\tilde{B}^{2,1} + \tilde{B}^{2,2})$
- $\gamma_7 = (\tilde{A}^{1,1} + \tilde{A}^{2,2})(\tilde{B}^{1,1} + \tilde{B}^{2,2})$.

The result matrix $\tilde{C} = \tilde{A}\tilde{B}$ has the form

$$\tilde{C} = \begin{bmatrix} c_{1,1} & c_{1,2} & c_{1,3} & 0 \\ c_{2,1} & c_{2,2} & c_{2,3} & 0 \\ c_{3,1} & c_{3,2} & c_{3,3} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix};$$

its block decomposition is

$$\tilde{C}^{1,1} = \begin{bmatrix} c_{1,1} & c_{1,2} \\ c_{2,1} & c_{2,2} \end{bmatrix}, \quad \tilde{C}^{1,2} = \begin{bmatrix} c_{1,3} & 0 \\ c_{2,3} & 0 \end{bmatrix}, \quad \tilde{C}^{2,1} = \begin{bmatrix} c_{3,1} & c_{3,2} \\ 0 & 0 \end{bmatrix}, \quad \tilde{C}^{2,2} = \begin{bmatrix} c_{3,3} & 0 \\ 0 & 0 \end{bmatrix},$$

whose entries are given by

- $\tilde{C}^{1,1} = \gamma_1 + \gamma_6 + \gamma_7 - \gamma_4$
- $\tilde{C}^{1,2} = \gamma_2 + \gamma_4$
- $\tilde{C}^{2,1} = \gamma_1 + \gamma_3$
- $\tilde{C}^{2,2} = \gamma_2 - \gamma_3 + \gamma_5 + \gamma_7$.

Naively, all $\gamma_i$ can be computed using 7 multiplications, using Strassen's algorithm recursively. However, some may visibly be done in smaller size than $2 \times 2$: this just amounts to remember that we have put zeros in the matrix. Consider for instance $\gamma_1 = \tilde{A}^{2,2}(\tilde{B}^{2,1} - \tilde{B}^{1,1})$. Since $\tilde{A}^{2,2}$ has one row full of zeros, we can reduce the cost from 7 to 4 multiplications.

However, better can be done, by noticing that even if some $\gamma_i$ is indeed of size $2 \times 2$, we may not need all of it for the end result. Consider for instance $\gamma_5 = (\tilde{A}^{2,1} - \tilde{A}^{1,1})(\tilde{B}^{1,1} + \tilde{B}^{1,2})$. There is no zero row or column in any of the terms in this product. However, $\gamma_5$ is used only to compute $\tilde{C}^{2,2} = \gamma_2 - \gamma_3 + \gamma_5 + \gamma_7$, and we know that $\tilde{C}^{2,2}$ has only one non-zero term, so that we only need one term in $\gamma_5$: we reduce the cost from 7 to 2 multiplications.

## 4.3 The general case

In this section, we describe a generalization of the remarks made above. We will show how to read off the presence of zeros in the input or output blocks from the multiplication pattern we are using.

First, we need to introduce a function to resize a matrix, by increasing or reducing its dimensions. Given a matrix $Z$ of size $(g, h)$ and integers $\alpha, \beta$, the matrix

$$Z' = \mathsf{R}(Z, \alpha, \beta)$$

is the matrix of size $(\alpha, \beta)$ defined as follows:

- If $\alpha \leq g$ then omit the rows in $Z$ of indices greater than $\alpha$; if $\alpha > g$ then pad $\alpha - g$ zero rows at the bottom of $Z$, to make the row dimension equal to $\alpha$.

- If $\beta \leq h$ then omit the columns in $Z$ of indices greater than $\beta$; if $\beta > h$ then pad $\beta - h$ rightmost zero columns to make the column dimension equal to $\beta$.

Given a problem size $\langle m, n, p\rangle$ and a pattern $(U_k, V_k, W_k)_{k\leq L}$ of format $(a, b, c)$, our question is to use the pattern to perform multiplication in size $(m, n) \times (n, p)$ (in the former example, the problem size was $m = n = p = 3$ and the pattern was Strassen's).

To solve the subdivision issue, we need some extra information. We will also require the following:

- subdivisions $\mathbf{m} = (m_1, \ldots, m_a)$ of $m$, $\mathbf{n} = (n_1, \ldots, n_b)$ of $n$ and $\mathbf{p} = (p_1, \ldots, p_c)$ of $p$;
- integers $(\mu_k)_{k\leq L}$, $(\nu_k)_{k\leq L}$ and $(\pi_k)_{k\leq L}$.

The integers $\mathbf{m}$, $\mathbf{n}$ and $\mathbf{p}$ give us the sizes of the submatrices in the matrices $A$, $B$ and $C$; the integers $\mu_k$, $\nu_k$ and $\pi_k$ tell us in what size we perform the $k$th linear combination. Precisely, if we subdivide $A, B, C$ using $\mathbf{m}$, $\mathbf{n}$ and $\mathbf{p}$, we obtain

$$A = \begin{bmatrix} A^{1,1} & \cdots & A^{1,b} \\ \vdots & & \vdots \\ A^{a,1} & \cdots & A^{a,b} \end{bmatrix}, \quad N = \begin{bmatrix} B^{1,1} & \cdots & B^{1,c} \\ \vdots & & \vdots \\ B^{b,1} & \cdots & B^{b,c} \end{bmatrix};$$

here $A^{i,j}$ has size $(m_i, n_j)$ and $B^{i,j}$ has size $(n_i, p_j)$; the product $C$ has the form

$$C = \begin{bmatrix} C^{1,1} & \cdots & C^{1,c} \\ \vdots & & \vdots \\ C^{a,1} & \cdots & C^{a,c} \end{bmatrix},$$

where $C^{i,j}$ has size $(m_i, p_j)$.

Next, we explain how to perform the multiplication. To obtain equally sized matrices in the linear combination, we apply the resize function, using the integers $\mu_k$, $\nu_k$ and $\pi_k$. That is, we compute, for $k = 1, \ldots, L$

- $\alpha_k = \sum_{i\leq a} \sum_{j\leq b} u_{k,i,j} \tilde{A}^{k,i,j}$, with $\tilde{A}^{k,i,j} = \mathsf{R}(A^{i,j}, \mu_k, \nu_k)$; note that $\alpha_k$ has size $(\mu_k, \nu_k)$
- $\beta_k = \sum_{i\leq b} \sum_{j\leq c} v_{k,i,j} \tilde{B}^{k,i,j}$, with $\tilde{B}^{k,i,j} = \mathsf{R}(B^{i,j}, \nu_k, \pi_k)$; note that $\beta_k$ has size $(\nu_k, \pi_k)$
- $\gamma_k = \alpha_k \beta_k$; note that $\gamma_k$ has size $(\mu_k, \pi_k)$

Finally, for $i \leq a$ and $j \leq b$, we compute $\tilde{C}^{i,j} = \sum_{k \leq L} w_{k,i,j} \mathsf{R}(\gamma_k, m_i, p_j)$, and we assemble $\tilde{C}$ from the blocks $\tilde{C}^{i,j}$.

There is no guarantee that such an algorithm produces the correct result: if $\mu_k, \nu_k, \pi_k$ are too small, the products $\gamma_k$ will not contain enough information. We are now going to give a sufficient condition to ensure validity. To do so, we need the following quantities, for $k = 1, \ldots, L$:

$$
\begin{aligned}
R_{1,k} &= \max\{m_i \text{ for } i \leq a, j \leq b \text{ such that } u_{k,i,j} \neq 0\} && (4.1)\\
C_{1,k} &= \max\{n_i \text{ for } i \leq a, j \leq b \text{ such that } u_{k,i,j} \neq 0\} && (4.2)\\
R_{2,k} &= \max\{n_i \text{ for } i \leq b, j \leq c \text{ such that } v_{k,i,j} \neq 0\} && (4.3)\\
C_{2,k} &= \max\{p_i \text{ for } i \leq b, j \leq c \text{ such that } v_{k,i,j} \neq 0\} && (4.4)\\
R_{3,k} &= \max\{m_i \text{ for } i \leq a, j \leq c \text{ such that } w_{k,i,j} \neq 0\} && (4.5)\\
C_{3,k} &= \max\{p_i \text{ for } i \leq a, j \leq c \text{ such that } w_{k,i,j} \neq 0\}. && (4.6)
\end{aligned}
$$

Remark that these quantities depend on the sparseness of the pattern: the more zeros, the smaller they are. In particular, Strassen's pattern and its modification by Winograd may not give the same bounds, even though they have the same number of multiplications (and indeed, on some examples, the results differ).

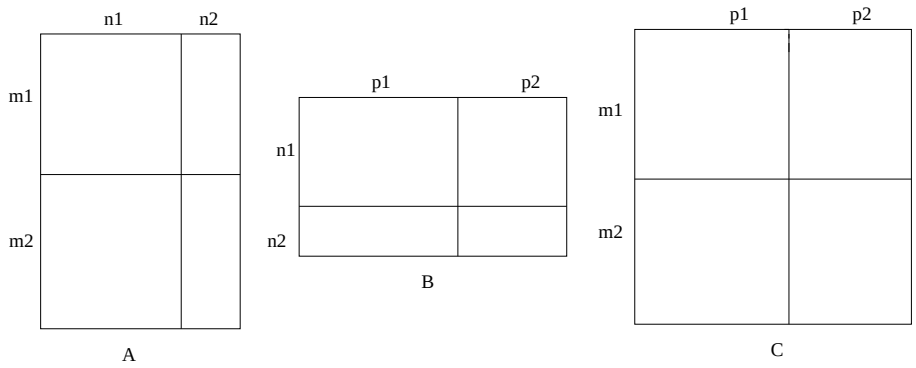
Then, the following proposition gives a lower bound on the integers $\mu_k$, $\nu_k$ and $\pi_k$ which ensures correctness.

**Proposition 1.** *Suppose that the following conditions hold for all $k \leq L$:*

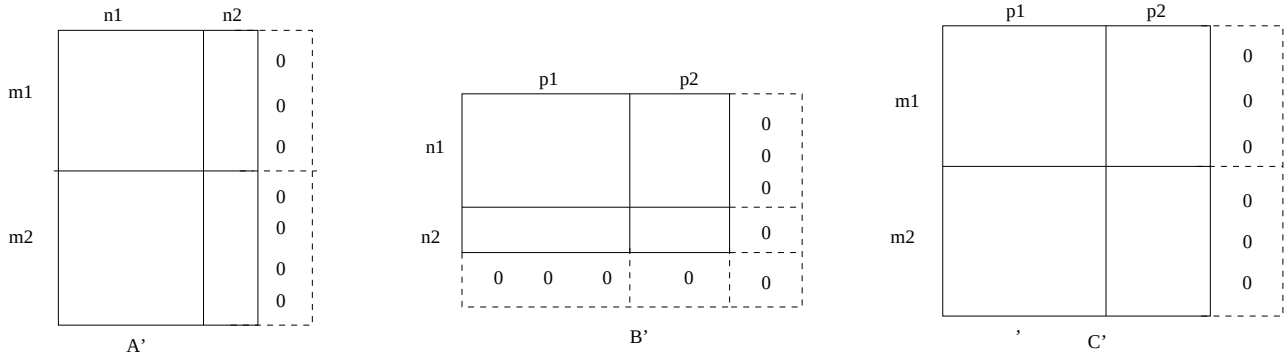
- $\mu_k \geq \min(R_{1,k}, R_{3,k})$,
- $\nu_k \geq \min(C_{1,k}, R_{2,k})$,
- $\pi_k \geq \min(C_{2,k}, C_{3,k})$,

*Then $\tilde{C} = AB$.*

*Proof.* We are given matrices $A$ and $B$, and perform their subdivision, as for instance in the following example with $(a, b, c) = (2, 2, 2)$ (so that here $\mathbf{m} = (m_1, m_2), \mathbf{n} = (n_1, n_2), \mathbf{p} = (p_1, p_2)$).




Let us also describe the classical way to multiply $A$ and $B$ by padding. Let $m' = \max(m_1, \ldots, m_a)$, $n' = \max(n_1, \ldots, n_b)$ and $p' = \max(p_1, \ldots, p_c)$. We add rightmost and bottom-most columns and rows of zeros in all submatrices $A^{i,j}$ and $B^{i,j}$ to obtain blocks $A'^{i,j}$ and $B'^{i,j}$ of sizes $(m', n')$ and $(n', p')$ respectively. Thus, we have $A'^{i,j} = \mathsf{R}(A^{i,j}, m', n')$ and $B'^{i,j} = \mathsf{R}(B^{i,j}, n', p')$. On our example, $m_1 = m_2$ so there is no need to pad rows in $A$, and we get




For these larger matrices, we are in the nice case where the pattern's format exactly divides the problem size, so we can apply the block multiplication described in Subsection 2.2.2. Explicitly, for $k = 1, \ldots, L$, we define

- $\alpha'_k = \sum_{i \leq a} \sum_{j \leq b} u_{k,i,j} A'^{i,j}$
- $\beta'_k = \sum_{i \leq b} \sum_{j \leq c} v_{k,i,j} B'^{i,j}$
- $\gamma'_k = \alpha'_k \beta'_k$

Then we let $C'^{i,j} = \sum_{k \leq L} w_{k,i,j} \gamma'_k$, so that the blocks in the result $C = AB$ are given by $C^{i,j} = \mathsf{R}(C'^{i,j}, m_i, p_j)$.

We have to prove that under the assumptions of the proposition, the output $\tilde{C}^{i,j}$ obtained in our algorithm agrees with the correct result $C^{i,j}$ we just obtained. First, we can rewrite $C^{i,j}$ as

$$C^{i,j} = \sum_{k \leq L, w_{k,i,j} \neq 0} w_{k,i,j} \mathsf{R}(\gamma'_k, m_i, p_j).$$

Adding the obvious conditions $u_{k,i,j} \neq 0$ and $v_{k,i,j} \neq 0$ in the definitions of $\alpha'$ and $\beta'$, we obtain after expansion

$$C^{i,j} \;=\; \sum_{k\leq L,\ w_{k,i,j}\neq 0} \;\; \sum_{i'\leq a,\ j'\leq b,\ u_{k,i',j'}\neq 0} \;\; \sum_{i''\leq b,\ j''\leq c,\ B_{k,i'',j''}\neq 0} T_{k,i,j,i',j',i'',j''}$$

with

$$T_{k,i,j,i',j',i'',j''} \;=\; u_{k,i',j'}\, v_{k,i'',j''}\, w_{k,i,j}\, \mathsf{R}(A'^{i',j'} B'^{i'',j''}, m_i, p_j).$$

Similarly, expanding the definition of $\tilde{C}^{i,j}$ gives

$$\tilde{C}^{i,j} \;=\; \sum_{k\leq L,\ w_{k,i,j}\neq 0} \;\; \sum_{i'\leq a,\ j'\leq b,\ u_{k,i',j'}\neq 0} \;\; \sum_{i''\leq b,\ j''\leq c,\ v_{k,i'',j''}\neq 0} \tilde{T}_{k,i,j,i',j',i'',j''}$$

with

$$\tilde{T}_{k,i,j,i',j',i'',j''} \;=\; u_{k,i',j'}\, v_{k,i'',j''}\, w_{k,i,j} \mathsf{R}(\tilde{A}^{k,i',j'} \tilde{B}^{k,i'',j''}, m_i, p_j).$$

To prove correctness, it is enough to prove that for all terms appearing, we have

$$\mathsf{R}(A'^{i',j'} B'^{i'',j''}, m_i, p_j) = \mathsf{R}(\tilde{A}^{k,i',j'} \tilde{B}^{k,i'',j''}, m_i, p_j).$$

We are going to simplify the left-hand side. The definitions of the matrices $A'^{i',j'}$ and $B'^{i'',j''}$ give

$$\mathsf{R}(A'^{i',j'} B'^{i'',j''}, m_i, p_j) = \mathsf{R}(\mathsf{R}(A^{i',j'}, m', n')\mathsf{R}(B^{i'',j''}, n', p'), m_i, p_j).$$

Since by assumption $m_i \leq m'$ and $p_j \leq p'$, we can rewrite this as

$$\mathsf{R}(A'^{i',j'} B'^{i'',j''}, m_i, p_j) = \mathsf{R}(A^{i',j'}, m_i, n')\mathsf{R}(B^{i'',j''}, n', p_j).$$

Let us do the same for the right-hand side. This will require more work. First, a substitution gives

$$\mathsf{R}(\tilde{A}^{k,i',j'} \tilde{B}^{k,i'',j''}, m_i, p_j) = \mathsf{R}(\mathsf{R}(A^{i',j'}, \mu_k, \nu_k)\mathsf{R}(B^{i'',j''}, \nu_k, \pi_k), m_i, p_j).$$

The assumption on $\mu_k$ shows that $\mu_k \geq \min(m_i, m_{i'})$. If $m_i \geq m_{i'}$, we obtain $\mu_k \geq m_{i'}$,

so the former product becomes

$$\mathsf{R}(\mathsf{R}(A^{i',j'}, m_i, \nu_k)\mathsf{R}(B^{i'',j''}, \nu_k, \pi_k), m_i, p_j).$$

If $m_{i'} \geq m_i$, we obtain $\mu_k \geq m_i$, so again, we obtain the form

$$\mathsf{R}(\mathsf{R}(A^{i',j'}, m_i, \nu_k)\mathsf{R}(B^{i'',j''}, \nu_k, \pi_k), m_i, p_j).$$

We proceed similarly on $\pi_k$, so that eventually we have

$$\mathsf{R}(\tilde{A}^{k,i',j'}\tilde{B}^{k,i'',j''}, m_i, p_j) = \mathsf{R}(\mathsf{R}(A^{i',j'}, m_i, \nu_k)\mathsf{R}(B^{i'',j''}, \nu_k, p_j), m_i, p_j);$$

this can now be simplified as

$$\mathsf{R}(\tilde{A}^{k,i',j'}\tilde{B}^{k,i'',j''}, m_i, p_j) = \mathsf{R}(A^{i',j'}, m_i, \nu_k)\mathsf{R}(B^{i'',j''}, \nu_k, p_j).$$

To finish the proof, it suffices to observe that

$$\mathsf{R}(A^{i',j'}, m_i, n')\mathsf{R}(B^{i'',j''}, n', p_j) = \mathsf{R}(A^{i',j'}, m_i, \nu_k)\mathsf{R}(B^{i'',j''}, \nu_k, p_j),$$

since both $\nu_k$ and $n'$ are greater than or equal to $\min(n_{i'}, n_{i''})$. □

# Chapter 5

# Trilinear aggregating

## 5.1 Introduction

Trilinear aggregating refers to a technique due to Pan [21], that enables one to perform three matrix products simultaneously, and can be adapted to perform a single product.

Following Pan's work, several declinations of this idea have been proposed. Roughly speaking, such algorithms enable one to perform three products of sizes $\langle a, b, c\rangle$, $\langle b, c, a\rangle$ and $\langle c, a, b\rangle$ using

$$abc + \alpha(ab + ac + bc) + \beta(a + b + c) + \gamma$$

multiplications, for various constants $\alpha, \beta, \gamma$, under constraints such as $a, b, c$ even. Used to perform a single matrix multiplication of size $n$, we obtain algorithms with costs of the form

$$\frac{n^3 + \alpha' n^2 + \beta' n + \gamma'}{3},$$

for some other constants $\alpha', \beta', \gamma'$. Asymptotically, considering the leading term, we see that this reduces the number of multiplication by a factor of three compared to the naive algorithm. However, the extra trailing terms in the cost estimates are not negligible for small matrix sizes.

In this chapter, we recall the main idea behind this algorithm and present a new version that improves previous ones. The contents of Sections 5.2 to 5.4 are from [17]. Our improvements follow; they are summarized in the following proposition.

**Proposition 1.** *If $n$ is even, one can compute $n \times n$ matrix products using*

$$A(n) = \frac{n^3 + 12n^2 + 11n}{3}$$

*multiplications. If $n$ is odd and greater than 3, one can compute $n \times n$ matrix products using*

$$A(n) = \frac{n^3 + 15n^2 + 14n - 6}{3}.$$

*multiplications.*

The best previous result that we are aware of was [17]

$$\frac{n^3}{3} + \frac{1}{3}\min\{12n^2 + 17n, \frac{45}{4}n^2 + \frac{97}{2}n + 60\},$$

for $n$ even; we do not know of any previous mention of the case of $n$ odd.

## 5.2 Polynomial representation

The algorithms in this chapter involve complex formulas. As it turns out, a compact way to represent them is to rewrite them using a polynomial notation. This notation is actually widely used in textbooks such as [6], where it is introduced as a means to decompose tensors. Our presentation is technically simpler, but sufficient to our purposes.

This representation is simply an alternative way to represent matrix multiplication patterns. Consider matrices $A = [a_{i,j}]_{m \times n}$ and $B = [b_{i,j}]_{n \times p}$, and let $\mathbf{C} = [\mathbf{c}_{i,j}]_{p \times m}$ be variables. Consider also the polynomial

$$\mathbf{T}(A, B, \mathbf{C}) = \sum_{1 \leq i \leq m,\, 1 \leq j \leq n,\, 1 \leq k \leq p} a_{i,j} b_{j,k} \mathbf{c}_{k,i},$$

and remark that for all $i, j$, the coefficient of $\mathbf{c}_{j,i}$ is the $(i, j)$ entry of the product $C = AB$ (the fact that $\mathbf{c}_{k,i}$ appears in the sum instead of $\mathbf{c}_{i,k}$ is mostly due to historical reasons).

Thus, a *polynomial representation* of a matrix multiplication pattern in size $\langle m, n, p \rangle$, over a ring $R$, is an expression of the form

$$\sum_{k \leq L} \alpha_k(A) \beta_k(B) \pi_j(\mathbf{C}),$$

where:

- $\alpha_k(A)$ is a linear combination of $[a_{i,j}]_{m\times n}$, with coefficients in $R$;
- $\alpha_k(B)$ is a linear combination of $[b_{i,j}]_{n\times p}$, with coefficients in $R$;
- $\pi_k(\mathbf{C})$ is a linear combination of $[\mathbf{c}_{i,j}]_{m\times p}$, with coefficients in $R$;

such that

$$\sum_{k\leq L} \alpha_k(A)\beta_k(B)\pi_j(\mathbf{C}) = \mathbf{T}(A,B,\mathbf{C}).$$

Let us give first the example of the polynomial representation of Winograd's pattern:

$$\begin{aligned}
\mathbf{T}(A,B,\mathbf{C}) &= a_{1,1}b_{1,1}(\mathbf{c}_{1,1}+\mathbf{c}_{1,2}+\mathbf{c}_{1,3}+\mathbf{c}_{1,4}) \\
&+ a_{1,2}b_{2,1}\mathbf{c}_{1,1} \\
&+ (a_{1,1}+a_{1,2}-a_{2,1}-a_{2,2})b_{2,2}\mathbf{c}_{1,2} \\
&+ a_{2,2}(b_{1,1}-b_{1,2}-b_{2,1}+b_{2,2})(-\mathbf{c}_{2,1}) \\
&+ (a_{2,1}+a_{2,2})(-b_{1,1}+b_{1,2})(\mathbf{c}_{1,2}+\mathbf{c}_{2,2}) \\
&+ (a_{2,1}+a_{2,2}-a_{1,1})(b_{2,2}-b_{1,2}+b_{1,1})(\mathbf{c}_{1,2}+\mathbf{c}_{2,1}+\mathbf{c}_{2,2}) \\
&+ (a_{1,1}-a_{2,1})(-b_{1,2}+b_{2,2})(\mathbf{c}_{2,1}+\mathbf{c}_{2,2}).
\end{aligned}$$

The term $a_{1,1}b_{1,1}(\mathbf{c}_{1,1}+\mathbf{c}_{1,2}+\mathbf{c}_{1,3}+\mathbf{c}_{1,4})$ tells us that the product $a_{1,1}b_{1,1}$ contributes to the result, with coefficient 1 in $\mathbf{c}_{1,1}$, coefficient 1 in $\mathbf{c}_{1,2}$, and so on.

Any multiplication pattern $U_k, V_k, W_k$ can be put in polynomial representation, by leaving the linear combinations $\alpha_k$ and $\beta_k$ unchanged, and taking for $\pi_k$ the sum

$$\pi_k = \sum_{1\leq i\leq m,\, 1\leq j\leq p} w_{k,i,j}\mathbf{c}_{i,j};$$

the converse transformation from a polynomial representation to a multiplication pattern is straightforward. The advantage of the polynomial multiplication is the compact form it can take, for patterns that display some regularity.

## 5.3 Disjoint products

Suppose that we want to compute the three products

$$C = AB, \quad W = UV, \quad Z = XY,$$

all matrices having size $m$. Let $\mathbf{C} = [\mathbf{c}_{i,j}]_{m\times m}$, $\mathbf{W} = [\mathbf{w}_{i,j}]_{m\times m}$ and $\mathbf{Z} = [\mathbf{z}_{i,j}]_{m\times m}$ be variables; using the polynomial representation, this means that we want to compute the polynomials

$$\begin{aligned}
\mathbf{T}(A, B, \mathbf{C}) &= \sum_{1\le i,j,k\le m} a_{i,j} b_{j,k} \mathbf{c}_{k,i}, \\
\mathbf{T}(U, V, \mathbf{W}) &= \sum_{1\le i,j,k\le m} u_{i,j} v_{j,k} \mathbf{w}_{k,i}, \\
\mathbf{T}(X, Y, \mathbf{Z}) &= \sum_{1\le i,j,k\le m} x_{i,j} y_{j,k} \mathbf{z}_{k,i},
\end{aligned}$$

or, equivalently, their sum

$$\begin{aligned}
\mathbf{T}(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}) &= \mathbf{T}(A, B, \mathbf{C}) + \mathbf{T}(U, V, \mathbf{W}) + \mathbf{T}(X, Y, \mathbf{Z}) \\
&= \sum_{1\le i,j,k\le m} a_{i,j} b_{j,k} \mathbf{c}_{k,i} + \sum_{1\le i,j,k\le m} u_{i,j} v_{j,k} \mathbf{w}_{k,i} \\
&+ \sum_{1\le i,j,k\le m} x_{i,j} y_{j,k} \mathbf{z}_{k,i}.
\end{aligned}$$

After permuting the indices by one place in the second sum and two places in the third, we can rewrite this as

$$\mathbf{T}(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}) = \sum_{1\le i,j,k\le m} T_{i,j,k}(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}), \tag{5.1}$$

with

$$T_{i,j,k} = a_{i,j} b_{j,k} \mathbf{c}_{k,i} + u_{j,k} v_{k,i} \mathbf{w}_{i,j} + x_{k,i} y_{i,j} \mathbf{z}_{j,k}.$$

Pan's idea consists in replacing a term $T_{i,j,k}$, which involves three products, by a single product, denoted by $S_{i,j,k}$, up to some corrective terms. Define

$$\begin{aligned}
S_{i,j,k}(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}) &= (a_{i,j} + u_{j,k} + x_{k,i})(b_{j,k} + v_{k,i} + y_{i,j})(\mathbf{c}_{k,i} + \mathbf{w}_{i,j} + \mathbf{z}_{j,k}), \\
R_{i,j,k}(A, V, \mathbf{Z}) &= a_{i,j} v_{k,i} \mathbf{z}_{j,k}, \\
U_{i,j,k}(A, Y, \mathbf{C}, \mathbf{W}, \mathbf{Z}) &= a_{i,j} y_{i,j} (\mathbf{c}_{k,i} + \mathbf{w}_{i,j} + \mathbf{z}_{j,k}),
\end{aligned}$$

and let $\mathbf{S}$, $\mathbf{R}$ and $\mathbf{U}$ be the sums of the corresponding quantities over $1 \le i, j, k \le m$. In all our formulas, we will explicit the dependency in $A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}$ only when needed; a term such as $\mathbf{R}(U, Y, \mathbf{C})$ is obtained by replacing $(A, V, \mathbf{Z})$ by $(U, Y, \mathbf{X})$ in the definition, etc.

We will want to rewrite the sum in (5.1) in terms of $\mathbf{R}$, $\mathbf{S}$ and $\mathbf{U}$. This is not always possible, but turns out to be so in the particular case where all row sums and column sums are zero (otherwise, many extra terms will appear).

**Lemma 1.** *If the sums of all rows and columns of $A, B, U, V, X, Y$ are zero, then the following equality holds:*

$$\begin{aligned}\mathbf{T}(A,B,\mathbf{C},U,V,\mathbf{W},X,Y,\mathbf{Z}) &= \mathbf{S}(A,B,\mathbf{C},U,V,\mathbf{W},X,Y,\mathbf{Z})\\ &- \mathbf{R}(A,V,\mathbf{Z}) - \mathbf{R}(U,Y,\mathbf{C}) - \mathbf{R}(X,B,\mathbf{W})\\ &- \mathbf{U}(A,Y,\mathbf{C},\mathbf{W},\mathbf{Z}) - \mathbf{U}(U,B,\mathbf{W},\mathbf{Z},\mathbf{C})\\ &- \mathbf{U}(X,V,\mathbf{Z},\mathbf{C},\mathbf{W}).\end{aligned}$$

*Proof.* Expanding the product for $S_{i,j,k}$ shows that we have

$$\begin{aligned}T_{i,j,k} &= S_{i,j,k}\\ &- a_{i,j}b_{j,k}(\mathbf{w}_{i,j}+\mathbf{z}_{j,k}) - a_{i,j}v_{k,i}(\mathbf{c}_{k,i}+\mathbf{w}_{i,j}+\mathbf{z}_{j,k}) - a_{i,j}y_{i,j}(\mathbf{c}_{k,i}+\mathbf{w}_{i,j}+\mathbf{z}_{j,k})\\ &- u_{j,k}b_{j,k}(\mathbf{c}_{k,i}+\mathbf{w}_{i,j}+\mathbf{z}_{j,k}) - u_{j,k}v_{k,i}(\mathbf{c}_{k,i}+\mathbf{z}_{j,k}) - u_{j,k}y_{i,j}(\mathbf{c}_{k,i}+\mathbf{w}_{i,j}+\mathbf{z}_{j,k})\\ &- x_{k,i}b_{j,k}(\mathbf{c}_{k,i}+\mathbf{w}_{i,j}+\mathbf{z}_{j,k}) - x_{k,i}v_{k,i}(\mathbf{c}_{k,i}+\mathbf{w}_{i,j}+\mathbf{z}_{j,k}) - x_{k,i}y_{i,j}(\mathbf{c}_{k,i}+\mathbf{w}_{i,j}).\end{aligned}$$

Summing over all $i, j, k$, several terms will disappear. For instance, consider the group of terms $a_{i,j}b_{j,k}(\mathbf{w}_{i,j}+\mathbf{z}_{j,k})$ appearing in the previous equality. After summation, we obtain

$$\sum_{1\le i,j,k\le m} a_{i,j}b_{j,k}(\mathbf{w}_{i,j}+\mathbf{z}_{j,k}) = \sum_{1\le i,j,k\le m} a_{i,j}b_{j,k}\mathbf{w}_{i,j} + \sum_{1\le i,j,k\le m} a_{i,j}b_{j,k}\mathbf{z}_{j,k}.$$

The first sum becomes

$$\sum_{1\le i,j\le m} a_{i,j}\left(\sum_{1\le k\le m} b_{j,k}\right)\mathbf{w}_{i,j},$$

which vanishes because $\sum_{1\le k\le m} b_{j,k} = 0$ for all $j$; the second sum vanishes as well. Inspecting all terms, we obtain the result. □

## 5.4 Single product

We will now adapt the former idea to perform a single product. We suppose that the matrix sizes are of the form $2n$, and we make no assumption on zero sums for rows

or columns. We subdivide the input matrices $A, B$ into blocks of size $n$ as

$$A = \begin{bmatrix} A^{1,1} & A^{1,2} \\ A^{2,1} & A^{2,2} \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} B^{1,1} & B^{1,2} \\ B^{2,1} & B^{2,2} \end{bmatrix}.$$

Following [17], we are first going to replace each block of size $n$ by a block of size $n+1$ whose rows and columns add up to zero. Let $I$ be the identity matrix of size $n$, and let $u$ be the vector of length $n$ given by

$$u = \begin{bmatrix} 1 & \cdots & 1 \end{bmatrix}.$$

Then, we define the matrices of respective sizes $(n+1, n)$ and $(n, n+1)$

$$L = \begin{bmatrix} I \\ -u \end{bmatrix}, \quad R = \begin{bmatrix} I - \frac{1}{n+1} u^t u & -\frac{1}{n+1} u^t \end{bmatrix};$$

if needed, we will write $L_n$ and $R_n$ to indicate these matrices' sizes. Next, we define new blocks of size $n+1$, $\tilde{A}^{i,j} = L\, A^{i,j}\, R$ and $\tilde{B}^{i,j} = L\, B^{i,j}\, L^t$. One can check that in all these new blocks, all columns and rows have zero sum. Finally, we define the matrices $\tilde{A}$ and $\tilde{B}$ of size $2n+2$ by

$$\tilde{A} = \begin{bmatrix} \tilde{A}^{1,1} & \tilde{A}^{1,2} \\ \tilde{A}^{2,1} & \tilde{A}^{2,2} \end{bmatrix} \quad \text{and} \quad \tilde{B} = \begin{bmatrix} \tilde{B}^{1,1} & \tilde{B}^{1,2} \\ \tilde{B}^{2,1} & \tilde{B}^{2,2} \end{bmatrix},$$

and compute $\tilde{C} = \tilde{A}\tilde{B}$. We can recover the result $C = AB$ from this larger product: decomposing $\tilde{C}$ into blocks of size $n+1$ as

$$\tilde{C} = \begin{bmatrix} \tilde{C}^{1,1} & \tilde{C}^{1,2} \\ \tilde{C}^{2,1} & \tilde{C}^{2,2} \end{bmatrix},$$

one can check that $C$ is given by

$$C = \begin{bmatrix} C^{1,1} & C^{1,2} \\ C^{2,1} & C^{2,2} \end{bmatrix},$$

where $C^{i,j}$ is obtained by resizing $\tilde{C}^{i,j}$ in size $(n, n)$. Since the blocks have rows and columns of zero sum, we will be able to apply the lemma of the previous section to compute $\tilde{C}$.

Hereafter, we write $m = n + 1$, and we define a few auxiliary polynomials. First,

for any matrices $U, V, \mathbf{W}$, we let

$$\mathbf{S}_0(U, V, \mathbf{W}) \;=\; \sum_{1 \le i \le m} u_{i,i} v_{i,i} \mathbf{w}_{i,i}.$$

Next, we consider the set

$$\mathcal{S}_1 \;=\; \{(i,j,k),\ 1 \le i \le j < k \le m \text{ or } 1 \le k < j \le i \le m\},$$

and we define

$$\begin{aligned} \mathbf{S}_1(U, V, \mathbf{W}) &= \sum_{(i,j,k) \in \mathcal{S}_1} S_{i,j,k}(U, V, \mathbf{W}, U, V, \mathbf{W}, U, V, \mathbf{W}) \\ &= \sum_{(i,j,k) \in \mathcal{S}_1} (u_{i,j} + u_{j,k} + u_{k,i})(v_{j,k} + v_{k,i} + v_{i,j})(\mathbf{w}_{k,i} + \mathbf{w}_{i,j} + \mathbf{w}_{j,k}). \end{aligned}$$

Finally, we let $\mathcal{S}_2'$ be the set

$$\mathcal{S}_2' \;=\; \{(i,j,k),\ 1 \le i,j,k \le m\} - \{(i,i,i),\ 1 \le i \le m\};$$

we define, for any matrices $A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}$,

$$\begin{aligned} \mathbf{S}_2(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}) &= \sum_{(i,j,k) \in \mathcal{S}_2'} S_{i,j,k}(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}) \\ &= \sum_{(i,j,k) \in \mathcal{S}_2'} (a_{i,j} + u_{j,k} + x_{k,i})(b_{j,k} + v_{k,i} + y_{i,j})(\mathbf{c}_{k,i} + \mathbf{w}_{i,j} + \mathbf{z}_{j,k}). \end{aligned}$$

Recall that we wish to compute the product $\tilde{C} = \tilde{A}\tilde{B}$; using the polynomial notation, we are to compute $\mathbf{T}(\tilde{A}, \tilde{B}, \tilde{\mathbf{C}})$, where $\tilde{\mathbf{C}}$ are variables that stand for the entries of $\tilde{C}$. The following proposition gives an expression for this sum; this result is taken from e.g. [17]. There is still room for improvement; this will be the subject of the next section.

**Proposition 2.** *The following equality holds:*

$$\begin{aligned}
\mathbf{T}(\tilde{A},\tilde{B},\tilde{\mathbf{C}}) &= 9\,\mathbf{S}_0(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) + \mathbf{S}_1(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) \\
&+ \mathbf{S}_0(\tilde{A}^{1,2}-\tilde{A}^{1,1}+\tilde{A}^{2,1},\ \tilde{B}^{2,1}+\tilde{B}^{1,2}+\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1}-\tilde{\mathbf{C}}^{1,2}+\tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{S}_2(\tilde{A}^{1,2},\ \tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,1},\ -\tilde{A}^{1,1},\ \tilde{B}^{1,2},-\tilde{\mathbf{C}}^{1,2},\ \tilde{A}^{2,1},\ \tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{S}_0(\tilde{A}^{1,2}+\tilde{A}^{2,1}-\tilde{A}^{2,2},\ \tilde{B}^{2,2}+\tilde{B}^{1,2}+\tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,2}+\tilde{\mathbf{C}}^{2,2}-\tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{S}_2(\tilde{A}^{1,2},\ \tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{A}^{2,1},\ \tilde{B}^{1,2},\ \tilde{\mathbf{C}}^{2,2},\ -\tilde{A}^{2,2},\ \tilde{B}^{2,1},-\tilde{\mathbf{C}}^{2,1}) \\
&+ 9\,\mathbf{S}_0(\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2}) + \mathbf{S}_1(\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2}) \\
&- \mathbf{U}(\ \tilde{A}^{1,1},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{\mathbf{C}}^{1,1}) \\
&- \mathbf{U}(\ \tilde{A}^{1,2},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},-\tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,1}) \\
&- \mathbf{U}(-\tilde{A}^{1,1},\tilde{B}^{2,1},-\tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,1}) \\
&- \mathbf{U}(\ \tilde{A}^{2,1},\tilde{B}^{1,2},\ \tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,1},-\tilde{\mathbf{C}}^{1,2}) \\
&- \mathbf{U}(\ \tilde{A}^{1,2},\tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,2},-\tilde{\mathbf{C}}^{2,1}) \\
&- \mathbf{U}(\ \tilde{A}^{2,1},\tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},-\tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,2}) \\
&- \mathbf{U}(-\tilde{A}^{2,2},\tilde{B}^{1,2},-\tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,2}) \\
&- \mathbf{U}(\ \tilde{A}^{2,2},\tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{\mathbf{C}}^{2,2}).
\end{aligned}$$

The rest of this section is devoted to the proof of this proposition: even though the result is not new, it is not widely known, and the proof provides an understanding of the origin of this complex formula.

We compute $\tilde{C} = \tilde{A}\tilde{B}$ using the naive $2\times 2$ block-algorithm, which uses 8 products of blocks. In other words, we have

$$\begin{aligned}
\mathbf{T}(\tilde{A},\tilde{B},\tilde{\mathbf{C}}) &= \mathbf{T}(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) + \mathbf{T}(\tilde{A}^{1,2},\tilde{B}^{2,1},\tilde{\mathbf{C}}^{1,1}) \\
&+ \mathbf{T}(\tilde{A}^{1,1},\tilde{B}^{1,2},\tilde{\mathbf{C}}^{1,2}) + \mathbf{T}(\tilde{A}^{1,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{1,2}) \\
&+ \mathbf{T}(\tilde{A}^{2,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{2,1}) + \mathbf{T}(\tilde{A}^{2,2},\tilde{B}^{2,1},\tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{T}(\tilde{A}^{2,1},\tilde{B}^{1,2},\tilde{\mathbf{C}}^{2,2}) + \mathbf{T}(\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2}).
\end{aligned}$$

We will next group some terms together, to use our formula for three disjoint multiplications. Observe first that we have

$$\mathbf{T}(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) = \frac{1}{3}\,\mathbf{T}(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1},\ \tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1},\ \tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1});$$

the same holds for $\mathbf{T}(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2})$. The remaining 6 products will be grouped together into two triples of disjoint products. To induce further simplifications, we change signs in some terms (without affecting the end result); concretely, we use the following expression:

$$\begin{aligned}
\mathbf{T}(\tilde{A}, \tilde{B}, \tilde{\mathbf{C}}) &= \frac{1}{3}\mathbf{T}(\tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}, \tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}, \tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}) \\
&+ \mathbf{T}(\tilde{A}^{1,2}, \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,1}, -\tilde{A}^{1,1}, \tilde{B}^{1,2}, -\tilde{\mathbf{C}}^{1,2}, \tilde{A}^{2,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{T}(\tilde{A}^{1,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{1,2}, \tilde{A}^{2,1}, \tilde{B}^{1,2}, \tilde{\mathbf{C}}^{2,2}, -\tilde{A}^{2,2}, \tilde{B}^{2,1}, -\tilde{\mathbf{C}}^{2,1}) \\
&+ \frac{1}{3}\mathbf{T}(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}, \tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}, \tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}).
\end{aligned} \tag{5.2}$$

We will apply Lemma 1 to each of the terms appearing in the right-hand side (this is valid, since all matrices have rows and columns with sum zero). Recall that Lemma 1 enables us to replace a $\mathbf{T}$ term by sums of terms of the form $\mathbf{S}$, $\mathbf{R}$ and $\mathbf{U}$. In our case, some further simplifications apply, which allow us to remove the $\mathbf{R}$ terms. Let us define

$$\begin{aligned}
\mathbf{T}'(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}) &= \mathbf{S}(A, B, \mathbf{C}, U, V, \mathbf{W}, X, Y, \mathbf{Z}) \\
&- \mathbf{U}(A, Y, \mathbf{C}, \mathbf{W}, \mathbf{Z}) - \mathbf{U}(U, B, \mathbf{W}, \mathbf{Z}, \mathbf{C}) \\
&- \mathbf{U}(X, V, \mathbf{Z}, \mathbf{C}, \mathbf{W}),
\end{aligned}$$

so that the $\mathbf{R}$ term has been omitted. The following lemma shows that removing these terms is harmless.

**Lemma 2.** *The following equality holds:*

$$\begin{aligned}
\mathbf{T}(\tilde{A}, \tilde{B}, \tilde{\mathbf{C}}) &= \frac{1}{3}\mathbf{T}'(\tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}, \tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}, \tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}) \\
&+ \mathbf{T}'(\tilde{A}^{1,2}, \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,1}, -\tilde{A}^{1,1}, \tilde{B}^{1,2}, -\tilde{\mathbf{C}}^{1,2}, \tilde{A}^{2,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{T}'(\tilde{A}^{1,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{1,2}, \tilde{A}^{2,1}, \tilde{B}^{1,2}, \tilde{\mathbf{C}}^{2,2}, -\tilde{A}^{2,2}, \tilde{B}^{2,1}, -\tilde{\mathbf{C}}^{2,1}) \\
&+ \frac{1}{3}\mathbf{T}'(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}, \tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}, \tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}).
\end{aligned}$$

*Proof.* It is enough to prove that all $\mathbf{R}$ terms that arise when applying Lemma 1 cancel each other. Applying that lemma to all $\mathbf{T}$ terms in (5.2), we obtain the following $\mathbf{R}$

terms:

$$-\mathbf{R}(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1})-\mathbf{R}(\tilde{A}^{1,2},\tilde{B}^{1,2},\tilde{\mathbf{C}}^{2,1})-\mathbf{R}(-\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1})-\mathbf{R}(\tilde{A}^{2,1},\tilde{B}^{2,1},-\tilde{\mathbf{C}}^{1,2})$$
$$-\mathbf{R}(\tilde{A}^{1,2},\tilde{B}^{1,2},-\tilde{\mathbf{C}}^{2,1})-\mathbf{R}(\tilde{A}^{2,1},\tilde{B}^{2,1},\tilde{\mathbf{C}}^{1,2})-\mathbf{R}(-\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2})-\mathbf{R}(\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2}),$$

which add up to zero, as needed. □

Expanding the definition of the terms $\mathbf{T}'$, we have thus obtained

$$\begin{aligned}
\mathbf{T}(\tilde{A},\tilde{B},\tilde{\mathbf{C}}) &= \frac{1}{3}\mathbf{S}(\tilde{A}^{1,1},\ \tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{A}^{1,1},\ \tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{A}^{1,1},\ \tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1})\\
&+ \mathbf{S}(\tilde{A}^{1,2},\ \tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,1},\ -\tilde{A}^{1,1},\ \tilde{B}^{1,2},-\tilde{\mathbf{C}}^{1,2},\ \tilde{A}^{2,1},\ \tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{2,1})\\
&+ \mathbf{S}(\tilde{A}^{1,2},\ \tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{A}^{2,1},\ \tilde{B}^{1,2},\ \tilde{\mathbf{C}}^{2,2},\ -\tilde{A}^{2,2},\ \tilde{B}^{2,1},-\tilde{\mathbf{C}}^{2,1})\\
&+ \frac{1}{3}\mathbf{S}(\tilde{A}^{2,2},\ \tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{A}^{2,2},\ \tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{A}^{2,2},\ \tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2})\\
&- \mathbf{U}(\ \tilde{A}^{1,1},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{\mathbf{C}}^{1,1})\\
&- \mathbf{U}(\ \tilde{A}^{1,2},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},-\tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,1})\\
&- \mathbf{U}(-\tilde{A}^{1,1},\tilde{B}^{2,1},\ -\tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,1})\\
&- \mathbf{U}(\ \tilde{A}^{2,1},\tilde{B}^{1,2},\ \tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,1},-\tilde{\mathbf{C}}^{1,2})\\
&- \mathbf{U}(\ \tilde{A}^{1,2},\tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,2},-\tilde{\mathbf{C}}^{2,1})\\
&- \mathbf{U}(\ \tilde{A}^{2,1},\tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},-\tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,2})\\
&- \mathbf{U}(-\tilde{A}^{2,2},\tilde{B}^{1,2},\ -\tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,2})\\
&- \mathbf{U}(\ \tilde{A}^{2,2},\tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{\mathbf{C}}^{2,2}). \qquad (5.3)
\end{aligned}$$

We next simplify the sums

$$\mathbf{S}(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1},\ \tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1},\ \tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1})$$

and

$$\mathbf{S}(\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2},\ \tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2},\ \tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2})$$

that arise at the first and fourth rows.

**Lemma 3.** *The following equality holds:*

$$\mathbf{S}(U,V,\mathbf{W},U,V,\mathbf{W},U,V,\mathbf{W})=27\mathbf{S}_0(U,V,\mathbf{W})+3\mathbf{S}_1(U,V,\mathbf{W}).$$

*Proof.* To complement the definition of the set $\mathcal{S}_1$ given before, we define

$$\begin{aligned}
\mathcal{S} &= \{(i,j,k),\ 1 \leq i,j,k \leq m\},\\
\mathcal{S}_0 &= \{(i,i,i),\ 1 \leq i \leq m\},\\
\mathcal{S}_1 &= \{(i,j,k),\ 1 \leq i \leq j < k \leq m \text{ or } 1 \leq k < j \leq i \leq m\},\\
\mathcal{S}_2 &= \{(j,k,i),(i,j,k) \in \mathcal{S}_1\}\\
&= \{(i,j,k),\ 1 \leq k \leq i < j \leq m \text{ or } 1 \leq j < i \leq k \leq m\},\\
\mathcal{S}_3 &= \{(k,i,j),(i,j,k) \in \mathcal{S}_1\}\\
&= \{(i,j,k),\ 1 \leq j \leq k < i \leq m \text{ or } 1 \leq i < k \leq j \leq m\}.
\end{aligned}$$

One checks that $\mathcal{S}$ is the disjoint union of $\mathcal{S}_0, \mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3$. Besides, for $\alpha = 0, 1, 2, 3$, we define

$$\mathbf{V}_\alpha(U,V,\mathbf{W}) = \sum_{(i,j,k)\in\mathcal{S}_\alpha} S_{i,j,k}(U,V,\mathbf{W},U,V,\mathbf{W},U,V,\mathbf{W}).$$

Remark that we have in particular

$$\mathbf{V}_0(U,V,\mathbf{W}) = 27\mathbf{S}_0(U,V,\mathbf{W}) \quad \text{and} \quad \mathbf{V}_i(U,V,\mathbf{W}) = \mathbf{S}_i(U,V,\mathbf{W}), \quad i > 0.$$

Besides, we also have

$$\mathbf{S}(U,V,\mathbf{W}) = \mathbf{V}_0(U,V,\mathbf{W}) + \mathbf{V}_1(U,V,\mathbf{W}) + \mathbf{V}_2(U,V,\mathbf{W}) + \mathbf{V}_3(U,V,\mathbf{W}),$$

by the disjointness of the decomposition. Finally, one checks the symmetry property

$$\mathbf{V}_1(U,V,\mathbf{W}) = \mathbf{V}_2(U,V,\mathbf{W}) = \mathbf{V}_3(U,V,\mathbf{W}).$$

Putting the previous considerations together concludes the proof. □

To finish the proof of Proposition 2, we apply Lemma 3 to the first and fourth rows in (5.3), and Lemma 4 below (whose proof is obvious from the definitions) to the second and third rows.

**Lemma 4.** *The following equality holds:*

$$\begin{aligned}
\mathbf{S}(A,B,\mathbf{C},U,V,\mathbf{W},X,Y,\mathbf{Z}) &= \mathbf{S}_2(A,B,\mathbf{C},U,V,\mathbf{W},X,Y,\mathbf{Z})\\
&+ \mathbf{S}_0(A+U+X, B+V+Y, \mathbf{C}+\mathbf{W}+\mathbf{Z}).
\end{aligned}$$

## 5.5 An improved version

The formula in Proposition 2 is almost in its final form, but some improvements are still possible.

The first one is straightforward. Remember that we are not interested in all of $\mathbf{T}(\tilde{A}, \tilde{B}, \tilde{\mathbf{C}})$: the product $\tilde{A}\tilde{B}$ has size $2m \times 2m = (2n+2) \times (2n+2)$, but we are only interested in four blocks of size $n \times n$. Hence, we can reduce the index sets used in our sums, using the following rule: any term of the form $\mathbf{c}_{a,b}^{i,j}$, with either $a = m$ or $b = m$, can be discarded.

For $\mathbf{S}_0$, the correction is straightforward: we replace the sum previously used by

$$\mathbf{s}_0(U, V, \mathbf{W}) \;=\; \sum_{1 \le i < m} u_{i,i} v_{i,i} \mathbf{w}_{i,i}.$$

To deal with $\mathbf{S}_1$, we replace $\mathcal{S}_1$ by the set

$$s_1 \;=\; \{(i,j,k),\ (i,j,k) \in \mathcal{S}_1 \text{ and } (i,j,k) \text{ contains at most one index equal to } m\}.$$

Then, we define

$$\mathbf{s}_1(U, V, \mathbf{W}) \;=\; \sum_{(i,j,k) \in s_1} (u_{i,j} + u_{j,k} + u_{k,i})(v_{j,k} + v_{k,i} + v_{i,j})(\mathbf{w}_{k,i} + \mathbf{w}_{i,j} + \mathbf{w}_{j,k});$$

indeed, for all terms of $\mathbf{S}_1$ left out of $\mathbf{s}_1$, all terms of $\mathbf{w}_{k,i} + \mathbf{w}_{i,j} + \mathbf{w}_{j,k}$ have one index equal to $m$. For the same reason, we replace $\mathcal{S}_2'$ by the set

$$s_2 \;=\; \{(i,j,k),\ (i,j,k) \in \mathcal{S}_2' \text{ and } (i,j,k) \text{ contains at most one index equal to } m\}$$

and we define

$$\mathbf{s}_2(A, B, \mathbf{C},\ U, V, \mathbf{W},\ X, Y, \mathbf{Z}) \;=\; \sum_{(i,j,k) \in s_2} (a_{i,j} + u_{j,k} + x_{k,i})(b_{j,k} + v_{k,i} + y_{i,j})(\mathbf{c}_{k,i} + \mathbf{w}_{i,j} + \mathbf{z}_{j,k}).$$

It is thus possible to replace all instances of $\mathbf{S}_0, \mathbf{S}_1, \mathbf{S}_2$ in Proposition 2 by $\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2$. Next, we are going to rewrite terms such as $\mathbf{U}(A, Y, \mathbf{C}, \mathbf{W}, \mathbf{Z})$. Recall that we have

$$\begin{aligned} \mathbf{U}(A, Y, \mathbf{C}, \mathbf{W}, \mathbf{Z}) \;&=\; \sum_{1 \le i,j,k \le m} a_{i,j} y_{i,j} (\mathbf{c}_{k,i} + \mathbf{w}_{i,j} + \mathbf{z}_{j,k}) \\ &=\; \sum_{1 \le i,j \le m} a_{i,j} y_{i,j} \sum_{1 \le k \le m} (\mathbf{c}_{k,i} + \mathbf{w}_{i,j} + \mathbf{z}_{j,k}). \end{aligned}$$

We split the summation set $\{(i,j),\ 1 \leq i,j \leq m\}$ for the outer sum into five subsets:

- $1 \leq i,j < m$ and $i \neq j$
- $1 \leq i,j < m$ and $i = j$
- $1 \leq i < m$ and $j = m$
- $1 \leq j < m$ and $i = m$
- $i = j = m$.

In the first two cases, the inner sum on $k$ ranges up to $m$; in the next two cases, it is enough to go up $m-1$, since in the remaining term is $\mathbf{w}_{i,m} + \mathbf{c}_{m,i} + \mathbf{z}_{m,m}$ (resp. $\mathbf{w}_{m,j} + \mathbf{c}_{m,m} + \mathbf{z}_{j,m}$). The fifth case can be dismissed altogether, for the same reason. Thus, we are led to define

$$\begin{aligned}
\mathbf{u}_1(A,Y,\mathbf{C},\mathbf{W},\mathbf{Z}) &= \sum_{1\leq i,j<m, i\neq j} a_{i,j}y_{i,j} \sum_{1\leq k\leq m} \mathbf{w}_{i,j} + \mathbf{c}_{k,i} + \mathbf{z}_{j,k} \\
\mathbf{u}_2(A,Y,\mathbf{C},\mathbf{W},\mathbf{Z}) &= \sum_{1\leq i<m} a_{i,i}y_{i,i} \sum_{1\leq k\leq m} \mathbf{w}_{i,i} + \mathbf{c}_{k,i} + \mathbf{z}_{i,k} \\
\mathbf{u}_3(A,Y,\mathbf{C}) &= \sum_{1\leq i<m} a_{i,m}y_{i,m} \sum_{1\leq k<m} \mathbf{c}_{k,i} \\
\mathbf{u}_4(A,Y,\mathbf{Z}) &= \sum_{1\leq j<m} a_{m,j}y_{m,j} \sum_{1\leq k<m} \mathbf{z}_{j,k}
\end{aligned}$$

and

$$\begin{aligned}
\mathbf{u}(A,Y,\mathbf{C},\mathbf{W},\mathbf{Z}) &= \mathbf{u}_1(A,Y,\mathbf{C},\mathbf{W},\mathbf{Z}) + \mathbf{u}_2(A,Y,\ \mathbf{C},\mathbf{W},\mathbf{Z}) \\
&+ \mathbf{u}_3(A,Y,\mathbf{C}) + \mathbf{u}_4(A,Y,\mathbf{Z}).
\end{aligned}$$

Remark that some terms have been omitted: all of them involved at least one index equal to $m$. Putting together the modifications performed up to now, we conclude

that it is enough to compute the following sum to be able to deduce the product $AB$:

$$\begin{aligned}
\mathbf{t}(\tilde{A},\tilde{B},\tilde{\mathbf{C}}) &= 9\,\mathbf{s}_0(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) + \mathbf{s}_1(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) \\
&+ \mathbf{s}_0(\tilde{A}^{1,2}-\tilde{A}^{1,1}+\tilde{A}^{2,1},\ \tilde{B}^{2,1}+\tilde{B}^{1,2}+\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1}-\tilde{\mathbf{C}}^{1,2}+\tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{s}_2(\tilde{A}^{1,2},\ \tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,1},\ -\tilde{A}^{1,1},\ \tilde{B}^{1,2},-\tilde{\mathbf{C}}^{1,2},\ \tilde{A}^{2,1},\ \tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{s}_0(\tilde{A}^{1,2}+\tilde{A}^{2,1}-\tilde{A}^{2,2},\ \tilde{B}^{2,2}+\tilde{B}^{1,2}+\tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,2}+\tilde{\mathbf{C}}^{2,2}-\tilde{\mathbf{C}}^{2,1}) \\
&+ \mathbf{s}_2(\tilde{A}^{1,2},\ \tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{A}^{2,1},\ \tilde{B}^{1,2},\ \tilde{\mathbf{C}}^{2,2},\ -\tilde{A}^{2,2},\ \tilde{B}^{2,1},-\tilde{\mathbf{C}}^{2,1}) \\
&+ 9\,\mathbf{s}_0(\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2}) + \mathbf{s}_1(\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2}) \\
&- \mathbf{u}(\ \tilde{A}^{1,1},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\ \tilde{\mathbf{C}}^{1,1}) \\
&- \mathbf{u}(\ \tilde{A}^{1,2},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},-\tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,1}) \\
&- \mathbf{u}(-\tilde{A}^{1,1},\tilde{B}^{2,1},\ -\tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,1}) \\
&- \mathbf{u}(\ \tilde{A}^{2,1},\tilde{B}^{1,2},\ \tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,1},-\tilde{\mathbf{C}}^{1,2}) \\
&- \mathbf{u}(\ \tilde{A}^{1,2},\tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,2},-\tilde{\mathbf{C}}^{2,1}) \\
&- \mathbf{u}(\ \tilde{A}^{2,1},\tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},-\tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,2}) \\
&- \mathbf{u}(-\tilde{A}^{2,2},\tilde{B}^{1,2},\ -\tilde{\mathbf{C}}^{2,1},\ \tilde{\mathbf{C}}^{1,2},\ \tilde{\mathbf{C}}^{2,2}) \\
&- \mathbf{u}(\ \tilde{A}^{2,2},\tilde{B}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{\mathbf{C}}^{2,2},\ \tilde{\mathbf{C}}^{2,2}).
\end{aligned}$$

Introducing the sums $\mathbf{u}_1,\ldots,\mathbf{u}_4$ in this sum gives in particular terms such as

$$\begin{aligned}
\mathbf{u}_2(\tilde{A}^{1,1},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\tilde{\mathbf{C}}^{1,1},\tilde{\mathbf{C}}^{1,1}) &= \sum_{1\le i<m} a_{i,i}^{1,1}b_{i,i}^{1,1}\sum_{1\le k\le m} c_{i,i}^{1,1}+c_{k,i}^{1,1}+c_{i,k}^{1,1} \\
&= \sum_{1\le i<m} a_{i,i}^{1,1}b_{i,i}^{1,1}\left(mc_{i,i}^{1,1}+\sum_{1\le k\le m} c_{k,i}^{1,1}+c_{i,k}^{1,1}\right).
\end{aligned}$$

Now, we can remark that the following sub-sum appears:

$$9\mathbf{s}_0(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) - \mathbf{u}_2(\tilde{A}^{1,1},\tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1},\tilde{\mathbf{C}}^{1,1},\tilde{\mathbf{C}}^{1,1}),$$

with

$$9\mathbf{s}_0(\tilde{A}^{1,1},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1}) = \sum_{1\le i<m} 9a_{i,i}^{1,1}b_{i,i}^{1,1}c_{i,i}^{1,1}.$$

Thus, we can group the terms as $-\mathbf{u}_2'(\tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1})$, with

$$\mathbf{u}_2'(\tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}) = \sum_{1 \le i < m} a_{i,i}^{1,1} b_{i,i}^{1,1} \left( (m-9) c_{i,i}^{1,1} + \sum_{1 \le k \le m} c_{k,i}^{1,1} + c_{i,k}^{1,1} \right).$$

The same remark holds for $\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}$, and shows that we can compute

$$9\mathbf{s}_0(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}) - \mathbf{u}_2(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}, \tilde{\mathbf{C}}^{2,2}, \tilde{\mathbf{C}}^{2,2})$$

as $-\mathbf{u}_2'(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2})$. The final simplification comes from terms of the form $\mathbf{u}_3$ and $\mathbf{u}_4$. A quick verification shows that we can compute the sum of all $\mathbf{u}_3$ terms as

$$\begin{aligned} &\mathbf{u}_3(\tilde{A}^{1,1} + \tilde{A}^{1,2}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}) + \mathbf{u}_3(\tilde{A}^{1,1} + \tilde{A}^{1,2}, \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,2}) + \\ &\quad \mathbf{u}_3(\tilde{A}^{2,1} + \tilde{A}^{2,2}, \tilde{B}^{1,2}, \tilde{\mathbf{C}}^{2,1}) + \mathbf{u}_3(\tilde{A}^{2,1} + \tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2}), \end{aligned}$$

and similarly, all $\mathbf{u}_4$ terms can be grouped as

$$\begin{aligned} &\mathbf{u}_4(\tilde{A}^{1,1}, \tilde{B}^{1,1} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,1}) + \mathbf{u}_4(\tilde{A}^{1,2}, \tilde{B}^{1,1} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{2,1}) + \\ &\quad \mathbf{u}_4(\tilde{A}^{2,1}, \tilde{B}^{2,2} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,2}) + \mathbf{u}_4(\tilde{A}^{2,2}, \tilde{B}^{2,2} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{2,2}). \end{aligned}$$

We have now obtained the final version of the multiplication algorithm. We review all components that appear, and count multiplications.

- $\mathbf{s}_1(\tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1})$
  $\mathbf{s}_1(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2})$
  each term uses $|s_1|$ multiplications, with $|s_1| = (m^3 - m)/3 - (m-1) = (m^3 - 4m + 3)/3$.

- $\mathbf{s}_0(\tilde{A}^{1,2} - \tilde{A}^{1,1} + \tilde{A}^{2,1}, \tilde{B}^{2,1} + \tilde{B}^{1,2} + \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1} - \tilde{\mathbf{C}}^{1,2} + \tilde{\mathbf{C}}^{2,1})$
  $\mathbf{s}_0(\tilde{A}^{1,2} + \tilde{A}^{2,1} - \tilde{A}^{2,2}, \tilde{B}^{2,2} + \tilde{B}^{1,2} + \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,2} + \tilde{\mathbf{C}}^{2,2} - \tilde{\mathbf{C}}^{2,1})$
  each term uses $m - 1$ multiplications.

- $\mathbf{s}_2(\tilde{A}^{1,2}, \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,1}, -\tilde{A}^{1,1}, \tilde{B}^{1,2}, -\tilde{\mathbf{C}}^{1,2}, \tilde{A}^{2,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{2,1})$
  $\mathbf{s}_2(\tilde{A}^{1,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{1,2}, \tilde{A}^{2,1}, \tilde{B}^{1,2}, \tilde{\mathbf{C}}^{2,2}, -\tilde{A}^{2,2}, \tilde{B}^{2,1}, -\tilde{\mathbf{C}}^{2,1})$
  each term uses $|s_2|$ multiplications, with $|s_2| = 3|s_1| = m^3 - 4m + 3$.

- $-\mathbf{u}_1(\tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}, \tilde{\mathbf{C}}^{1,1}, \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_1(\tilde{A}^{1,2}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2}, \tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_1(-\tilde{A}^{1,1}, \tilde{B}^{2,1}, -\tilde{\mathbf{C}}^{1,2}, \tilde{\mathbf{C}}^{2,1}, \tilde{\mathbf{C}}^{1,1})$

$-\mathbf{u}_1(\ \ \tilde{A}^{2,1}, \tilde{B}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2})$
$-\mathbf{u}_1(\ \ \tilde{A}^{1,2}, \tilde{B}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1})$
$-\mathbf{u}_1(\ \ \tilde{A}^{2,1}, \tilde{B}^{2,2}, \ \ \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,2})$
$-\mathbf{u}_1(-\tilde{A}^{2,2}, \tilde{B}^{1,2}, -\tilde{\mathbf{C}}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,2})$
$-\mathbf{u}_1(\ \ \tilde{A}^{2,2}, \tilde{B}^{2,2}, \ \ \tilde{\mathbf{C}}^{2,2}, \ \ \tilde{\mathbf{C}}^{2,2}, \ \ \tilde{\mathbf{C}}^{2,2})$
each term uses $(m-1)^2 - (m-1)$ multiplications.

- $-\mathbf{u}_2'(\ \ \tilde{A}^{1,1}, \tilde{B}^{1,1}, \ \ \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{1,2}, \tilde{B}^{1,1}, \ \ \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_2(-\tilde{A}^{1,1}, \tilde{B}^{2,1}, -\tilde{\mathbf{C}}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{2,1}, \tilde{B}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{1,2}, \tilde{B}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{2,1}, \tilde{B}^{2,2}, \ \ \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_2(-\tilde{A}^{2,2}, \tilde{B}^{1,2}, -\tilde{\mathbf{C}}^{2,1}, \ \ \tilde{\mathbf{C}}^{1,2}, \ \ \tilde{\mathbf{C}}^{2,2})$
  $-\mathbf{u}_2'(\ \ \tilde{A}^{2,2}, \tilde{B}^{2,2}, \ \ \tilde{\mathbf{C}}^{2,2})$
  each term uses $m-1$ multiplications.

- $-\mathbf{u}_3(\tilde{A}^{1,1} + \tilde{A}^{1,2}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_3(\tilde{A}^{1,1} + \tilde{A}^{1,2}, \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_3(\tilde{A}^{2,1} + \tilde{A}^{2,2}, \tilde{B}^{1,2}, \tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_3(\tilde{A}^{2,1} + \tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2})$
  each term uses $m-1$ multiplications.

- $-\mathbf{u}_4(\tilde{A}^{1,1}, \tilde{B}^{1,1} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_4(\tilde{A}^{1,2}, \tilde{B}^{1,1} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_4(\tilde{A}^{2,1}, \tilde{B}^{2,2} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_4(\tilde{A}^{2,2}, \tilde{B}^{2,2} - \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{2,2})$
  each term uses $m-1$ multiplications.

Summing, we obtain $(8m^3 + 24m^2 - 50m + 18)/3$ multiplications. Remember that $m = n+1$, and that we are multiplying matrices $A, B$ of size $2n$. Thus, to obtain the cost for multiplication in size $n$, with $n$ even, we replace $m$ by $n/2+1$ in the previous sum, and we finally obtain a cost of

$$\frac{n^3 + 12n^2 + 11n}{3}$$

multiplications.

## 5.6 The odd case

Finally, we discuss the extension of the previous construction to the case where the matrix size is odd, of the form $2n+1$. To our knowledge, no previous mention of the optimizations arising in this case appeared before. We still split the input matrices $A$ and $B$ into four blocks, which are now

$$A^{1,1}_{(n+1)\times(n+1)}, \quad A^{1,2}_{(n+1)\times n}, \quad A^{2,1}_{n\times(n+1)}, \quad A^{2,2}_{n\times n},$$

and similarly for $B$. The matrices $\tilde{A}^{i,j}$ and $\tilde{B}^{i,j}$ are defined as before, with now

$$\tilde{A}^{i,j} = L_i \, A^{i,j} \, R_i \quad \text{and} \quad \tilde{B}^{i,j} = L_i \, B^{i,j} \, L_j^t$$

and $L_1 = L_{n+1}, L_2 = L_n, R_1 = L_{n+1}$ and $R_2 = L_n$. Let $m = n+2$; then the sizes of these matrices are as follows:

- $\tilde{A}^{1,1}$ and $\tilde{B}^{1,1}$ have size $m \times m$
- $\tilde{A}^{1,2}$ and $\tilde{B}^{1,2}$ have size $m \times (m-1)$
- $\tilde{A}^{2,1}$ and $\tilde{B}^{2,1}$ have size $(m-1) \times m$
- $\tilde{A}^{2,2}$ and $\tilde{B}^{2,2}$ have size $(m-1) \times (m-1)$.

As before, we let $\tilde{C} = \tilde{A}\tilde{B}$, which we decompose into blocks

$$\tilde{C} = \begin{bmatrix} \tilde{C}^{1,1}_{m\times m} & \tilde{C}^{1,2}_{m\times(m-1)} \\ \tilde{C}^{2,1}_{(m-1)\times m} & \tilde{C}^{2,2}_{(m-1)\times(m-1)} \end{bmatrix}.$$

Again, $C = AB$ is given by

$$C = \begin{bmatrix} C^{1,1} & C^{1,2} \\ C^{2,1} & C^{2,2} \end{bmatrix},$$

where we discard the last row and column of all blocks $\tilde{C}^{i,j}$.

To use the strategy given in the previous sections to this case, we pad all blocks with zeros, to give them size $m \times m$. Thus, we add one zero column to $\tilde{A}^{1,2}$ and $\tilde{A}^{2,2}$, and one zero row to $\tilde{A}^{2,1}$ and $\tilde{A}^{2,2}$; we do the same to $\tilde{B}$. As it turns out, it is most efficient to insert this extra row / column, not at the last entry, but at the last-but-one: previous optimizations already reduced the number of operations involving the last row and column, so zeroing it would not be optimal.

With this choice, we review the various steps of the algorithm and indicate the possible savings:

- $\mathbf{s}_1(\tilde{A}^{1,1}, \tilde{B}^{1,1}, \tilde{\mathbf{C}}^{1,1})$
  $\mathbf{s}_1(\tilde{A}^{2,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{2,2})$
  no savings are apparent in the first sum. In the second one, one can dismiss all terms where two indices are equal to $m-1$; there are $m-1$ such terms.

- $\mathbf{s}_0(\tilde{A}^{1,2} - \tilde{A}^{1,1} + \tilde{A}^{2,1},\ \tilde{B}^{2,1} + \tilde{B}^{1,2} + \tilde{B}^{1,1},\ \tilde{\mathbf{C}}^{1,1} - \tilde{\mathbf{C}}^{1,2} + \tilde{\mathbf{C}}^{2,1})$
  $\mathbf{s}_0(\tilde{A}^{1,2} + \tilde{A}^{2,1} - \tilde{A}^{2,2},\ \tilde{B}^{2,2} + \tilde{B}^{1,2} + \tilde{B}^{2,1},\ \tilde{\mathbf{C}}^{1,2} + \tilde{\mathbf{C}}^{2,2} - \tilde{\mathbf{C}}^{2,1})$
  no savings are apparent in the first sum. In the second one, one can dismiss the term of index $m-1$.

- $\mathbf{s}_2(\tilde{A}^{1,2}, \tilde{B}^{2,1}, \tilde{\mathbf{C}}^{1,1},\ -\tilde{A}^{1,1}, \tilde{B}^{1,2}, -\tilde{\mathbf{C}}^{1,2},\quad \tilde{A}^{2,1}, \tilde{B}^{1,1},\quad \tilde{\mathbf{C}}^{2,1})$
  $\mathbf{s}_2(\tilde{A}^{1,2}, \tilde{B}^{2,2}, \tilde{\mathbf{C}}^{1,2},\quad \tilde{A}^{2,1}, \tilde{B}^{1,2},\quad \tilde{\mathbf{C}}^{2,2},\ -\tilde{A}^{2,2}, \tilde{B}^{2,1}, -\tilde{\mathbf{C}}^{2,1})$
  no savings are apparent in the first sum. In the second one, one can dismiss terms where two indices are equal to $m-1$; there are $3(m-1)$ such terms.

- $-\mathbf{u}_1(\ \ \tilde{A}^{1,1}, \tilde{B}^{1,1},\quad \tilde{\mathbf{C}}^{1,1},\quad \tilde{\mathbf{C}}^{1,1},\quad \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_1(\ \ \tilde{A}^{1,2}, \tilde{B}^{1,1},\quad \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_1(-\tilde{A}^{1,1}, \tilde{B}^{2,1},\ -\tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_1(\ \ \tilde{A}^{2,1}, \tilde{B}^{1,2},\quad \tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_1(\ \ \tilde{A}^{1,2}, \tilde{B}^{2,1},\quad \tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_1(\ \ \tilde{A}^{2,1}, \tilde{B}^{2,2},\quad \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_1(-\tilde{A}^{2,2}, \tilde{B}^{1,2},\ -\tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,2})$
  $-\mathbf{u}_1(\ \ \tilde{A}^{2,2}, \tilde{B}^{2,2},\quad \tilde{\mathbf{C}}^{2,2},\quad \tilde{\mathbf{C}}^{2,2},\quad \tilde{\mathbf{C}}^{2,2})$
  the presence of extra rows and columns of zeros reduces the cost of the second and third lines by $m-2$, and by $2(m-2)$ for all lines from the fourth on. In total, we save $12(m-2)$ multiplications.

- $-\mathbf{u}_2'(\ \ \tilde{A}^{1,1}, \tilde{B}^{1,1},\quad \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{1,2}, \tilde{B}^{1,1},\quad \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_2(-\tilde{A}^{1,1}, \tilde{B}^{2,1},\ -\tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{2,1}, \tilde{B}^{1,2},\quad \tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,1}, -\tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{1,2}, \tilde{B}^{2,1},\quad \tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_2(\ \ \tilde{A}^{2,1}, \tilde{B}^{2,2},\quad \tilde{\mathbf{C}}^{2,2}, -\tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_2(-\tilde{A}^{2,2}, \tilde{B}^{1,2},\ -\tilde{\mathbf{C}}^{2,1},\quad \tilde{\mathbf{C}}^{1,2},\quad \tilde{\mathbf{C}}^{2,2})$
  $-\mathbf{u}_2'(\ \ \tilde{A}^{2,2}, \tilde{B}^{2,2},\quad \tilde{\mathbf{C}}^{2,2})$

the presence of zeros saves one multiplication at each line, except the first one, for a total of 7.

- $-\mathbf{u}_3(\tilde{A}^{1,1}+\tilde{A}^{1,2},\tilde{B}^{1,1},\tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_3(\tilde{A}^{1,1}+\tilde{A}^{1,2},\tilde{B}^{2,1},\tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_3(\tilde{A}^{2,1}+\tilde{A}^{2,2},\tilde{B}^{1,2},\tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_3(\tilde{A}^{2,1}+\tilde{A}^{2,2},\tilde{B}^{2,2},\tilde{\mathbf{C}}^{2,2})$
  the presence of zeros saves one multiplication at each line, except the first one, for a total of 3.

- $-\mathbf{u}_4(\tilde{A}^{1,1},\tilde{B}^{1,1}-\tilde{B}^{2,1},\tilde{\mathbf{C}}^{1,1})$
  $-\mathbf{u}_4(\tilde{A}^{1,2},\tilde{B}^{1,1}-\tilde{B}^{2,1},\tilde{\mathbf{C}}^{2,1})$
  $-\mathbf{u}_4(\tilde{A}^{2,1},\tilde{B}^{2,2}-\tilde{B}^{2,1},\tilde{\mathbf{C}}^{1,2})$
  $-\mathbf{u}_4(\tilde{A}^{2,2},\tilde{B}^{2,2}-\tilde{B}^{2,1},\tilde{\mathbf{C}}^{2,2})$
  we save 3 multiplications, as in the previous case.

Without savings, the cost reported in the previous section was $(8m^3+24m^2-50m+18)/3$ multiplications. The savings add up to $16m-14$, whence a total of

$$\frac{8m^3+24m^2-98m+60}{3}$$

multiplications. To multiply matrices of odd size $n$, we actually have $m=(n-1)/2+2$, whence a total cost of

$$\frac{n^3+15n^2+14n-6}{3}.$$

# Chapter 6

# A generalization of Waksman's algorithm

## 6.1 Introduction

In 1969 Waksman [29] demonstrated an improvement on Winograd's algorithm [30] for the multiplication of two matrices with commutative entries. This improvement was given for square matrices; we present a generalized version of this algorithm which can be applied for both square and non-square cases of any size.

**Proposition 3.** *If $n$ is even, one can compute $\langle m, n, p\rangle$ matrix products over a commutative ring using*

$$mn'p + mn' + n'p - n'$$

*multiplications, with $n' = n/2$. If $n$ is odd, the number of multiplications is*

$$mn'p + mn' + n'p + pm - n',$$

*with $n' = (n-1)/2$.*

The number of multiplications required by our generalized version is the same as Waksman's when the matrices are square.

## 6.2 The algorithm

In all that follows, all matrices have entries in a commutative ring $R$. Let $A = [a_{i,j}]_{m\times n}$ and $B = [b_{i,j}]_{n\times p}$ be two such matrices, and let $n' = \lfloor n/2 \rfloor$. For $1 \leq \ell \leq m$, $1 \leq j \leq n'$

and $1 \leq k \leq p$, define

$$\begin{aligned} A_{\ell,j,k} &= (a_{\ell,2j-1} + b_{2j,k})(a_{\ell,2j} + b_{2j-1,k}) \\ B_{\ell,j,k} &= a_{\ell,2j-1}a_{\ell,2j} + b_{2j-1,k}b_{2j,k}; \end{aligned}$$

if $n$ is odd, let $c'_{\ell,k} = a_{\ell,n}b_{n,k}$, and let $c'_{\ell,k} = 0$ otherwise. Since $R$ is commutative, we obtain that

$$A_{\ell,j,k} - B_{\ell,j,k} = a_{\ell,2j-1}b_{2j-1,k} + a_{\ell,2j}b_{2j,k}.$$

Then, the entry $c_{\ell,k}$ of the product $C = AB$ is given by

$$c_{\ell,k} = \sum_{j=1}^{n'} (A_{\ell,j,k} - B_{\ell,j,k}) + c'_{\ell,k}.$$

This decomposition is the basis of Winograd's algorithm [30].

There are $mn'p$ terms of the form $A_{\ell,j,k}$; they will all be computed independently. Waksman's improvement is in the computation of the terms $B_{\ell,j,k}$. A direct count shows that one can compute all these terms using $mn' + n'p$ multiplications, by computing all terms $a_{\ell,2j-1}a_{\ell,2j}$ and $b_{2j-1,k}b_{2j,k}$ separately; Waksman's modification reduces the cost by $n'$.

Our presentation is slightly different from Waksman's, at it allows for a simple generalization when the matrices are not square. Remark that for all $\ell, \ell'$ and $k, k'$, with $j$ fixed, we have

$$B_{\ell,j,k} + B_{\ell',j,k'} = B_{\ell',j,k} + B_{\ell,j,k'},$$

so if three of the terms are known, we can deduce the fourth. In particular, suppose we know the "basic" terms $B_{1,j,k}$ and $B_{\ell,j,1}$ for all $\ell$ and $k$; then we deduce

$$B_{\ell,j,k} = B_{1,j,k} + B_{\ell,j,1} - B_{1,j,1}.$$

It remains to compute the basic terms. To do so, for $\ell, j, k$ as above, let

$$C_{\ell,j,k} = (a_{\ell,2j-1} - b_{2j,k})(a_{\ell,2j} - b_{2j-1,k});$$

then we can deduce

$$B_{\ell,j,k} = \frac{1}{2}(A_{\ell,j,k} + C_{\ell,j,k}).$$

Since there are $n'(m + p - 1)$ basic terms, and each requires one multiplication once $A_{\ell,j,k}$ is known, we arrive at a total of $n'(mp + m + p - 1)$ multiplications to compute

all $A_{\ell,j,k}$ and $B_{\ell,j,k}$; if $n$ is odd, we need another $mp$ multiplications to compute all $c'_{\ell,k}$. This proves our proposition.

Even though it is not the main focus of this thesis, it is worthwhile, and rather straightforward, to optimize additions as well. The details follow.

**Algorithm 1** `Generalized Waksman`

**Input:** two matrices $A = [a_{i,j}]_{m\times n}$ and $B = [b_{i,j}]_{n\times p}$
**Output:** the product $C = AB$

let $n' = \lfloor n/2 \rfloor$
let $C$ be an $m \times p$ matrix, initialized to 0
let $D$ be an array of length $p$, initialized to 0
let $E$ be an array of length $m$, initialized to 0
**for** $j = 1, \ldots, n'$ **do**
    **for** $k = 1, \ldots, p$ **do**
        $c_{1,k} = c_{1,k} + (a_{1,2j-1} + b_{2j,k})(a_{1,2j} + b_{2j-1,k})$
        $D_k = D_k + (a_{1,2j-1} - b_{2j,k})(a_{1,2j} - b_{2j-1,k})$
    **end for**
    **for** $\ell = 2, \ldots, m$ **do**
        $c_{\ell,1} = c_{\ell,1} + (a_{\ell,2j-1} + b_{2j,1})(a_{\ell,2j} + b_{2j-1,1})$
        $E_\ell = E_\ell + (a_{\ell,2j-1} - b_{2j,1})(a_{\ell,2j} - b_{2j-1,1})$
    **end for**
    **for** $k = 2, \ldots, p$ **do**
        **for** $\ell = 2, \ldots, m$ **do**
            $c_{\ell,k} = c_{\ell,k} + (a_{\ell,2j-1} + b_{2j,k})(a_{\ell,2j} + b_{2j-1,k})$
        **end for**
    **end for**
**end for**
$D_1 = (D_1 + c_{1,1})/2$
$c_{1,1} = c_{1,1} - D_1$
**for** $k = 2, \ldots, p$ **do**
    $D_k = (D_k + c_{1,k})/2$
    $c_{1,k} = c_{1,k} - D_k$
    $D_k = D_k - D_0$
**end for**
**for** $\ell = 2, \ldots, m$ **do**
    $E_\ell = (E_\ell + c_{\ell,1})/2$

```
    c_{ℓ,1} = c_{ℓ,1} − E_ℓ
end for
for k = 2, ..., p do
    for ℓ = 2, ..., m do
        c_{ℓ,k} = c_{ℓ,k} − D_k − E_ℓ
    end for
end for
if n is odd then
    for k = 1, ..., p do
        for ℓ = 1, ..., m do
            c_{ℓ,k} = c_{ℓ,k} + a_{ℓ,n} b_{n,k}
        end for
    end for
end if
```

# Chapter 7

# Results

## 7.1 Introduction

This chapter gives the results we obtain using our approach. These results are built into a database; for the moment, we only focus on lowering multiplication counts in this database. The implementation of the corresponding algorithms is described in the next chapter.

## 7.2 Database build-up

We use three main ingredients to build our database:

- block decomposition, relying on the family of patterns presented in Chapter 3, and using the alternative to peeling and padding presented in Chapter 4;
- Pan's idea of disjoint multiplications to group subproducts (when it helps);
- the algorithm of Chapter 5.

We used three algorithms, Optimizer, Subdivision and Tracker, which we describe now.

### 7.2.1 Optimizer

Given a pattern, a problem size $(m, n, p)$ and a subdivision $(\mathbf{m}, \mathbf{n}, \mathbf{p})$ of $(m, n, p)$, Optimizer determines the recursive calls to perform with this pattern, using the results of Chapter 4: this is the smallest possible choice of integers $(\mu_k, \nu_k, \pi_k)_{k \leq L}$ that satisfies the requirement of Proposition 1, to ensure correct results.

**Algorithm 2** `Optimizer`

**Input:**

- a product size $(m, n, p)$,
- a pattern $(A_k, B_k, C_k)_{k \leq L}$ of format $(a, b, c)$ and length $L$,
- a subdivision $(\mathbf{m}, \mathbf{n}, \mathbf{p})$ of $(m, n, p)$

**Output:** the sequence $(\mu_k, \nu_k, \pi_k)_{k \leq L}$ defined by

- $\mu_k = \min(R_{1,k}, R_{3,k})$,
- $\nu_k = \min(C_{1,k}, R_{2,k})$,
- $\pi_k = \min(C_{2,k}, C_{3,k})$,

where $R_{1,k}, \ldots, C_{3,k}$ are as in Equations (4.1), ..., (4.6).

### 7.2.2 List of patterns

For our optimization, we use patterns that describe algorithms given in Chapter 3: `Strassen`, `StrassenWinograd`, `Laderman`, `Hopcroft323`, `Hopcroft332`, `Hopcroft233`, `Makarov`. We also used three auxiliary patterns `add211`, `add121`, `add112`, defined as follows (their utility will be explained below).

- `add121` has length 2 and format $(1, 2, 1)$; it describes the product of a $(1, 2)$ matrix by a $(2, 1)$ matrix.

$$U = (\begin{bmatrix} 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \end{bmatrix}), \quad V = (\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}), \quad W = (\begin{bmatrix} 1 \end{bmatrix}, \begin{bmatrix} 1 \end{bmatrix}).$$

- `add211` has length 2 and format $(2, 1, 1)$

$$U = (\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}), \quad V = (\begin{bmatrix} 1 \end{bmatrix}, \begin{bmatrix} 1 \end{bmatrix}), \quad W = (\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix});$$

it simply describes the product of a $(2, 1)$ matrix by a $(1, 1)$ matrix.

- `add112` has length 2 and format $(1, 1, 2)$; it describes the product of a $(1, 1)$ matrix by a $(1, 2)$ matrix.

$$U = (\begin{bmatrix}1\end{bmatrix}, \begin{bmatrix}1\end{bmatrix}), \quad V = (\begin{bmatrix}1 & 0\end{bmatrix}, \begin{bmatrix}0 & 1\end{bmatrix}), \quad W = (\begin{bmatrix}1 & 0\end{bmatrix}, \begin{bmatrix}0 & 1\end{bmatrix}).$$

Applying `add121` to a problem $(m, n, p)$ simply amounts to cut $n$ into $n = n_1 + n_2$ and process the two parts independently; if for instance $n_2 = 1$, we recover the peeling rule that separates one column. More generally, these patterns provide us with the equivalent to the (A) rule of Probert and Fischer [22], described in Chapter 3.

All these patterns are stored in files, which simply list the entries of the respective matrices $(U_k, V_k, W_k)$.

### 7.2.3 Finding appropriate subdivisions

For a given pattern of format $(a, b, c)$ and a given problem size $(m, n, p)$, there are too many subdivisions $(\mathbf{m}, \mathbf{n}, \mathbf{p})$ as $m = m_1 + \cdots + m_a$, $n = n_1 + \cdots + n_b$ and $p = p_1 + \cdots + p_c$, for us to try them all when $(m, n, p)$ get too large.

We tried all possible subdivisions for various problem sizes, to determine where optimal upper bounds were obtained. After trying up to a meaningful range, $(12, 12, 12)$, we found that the optimums are obtained for

$$m/a \leq m_i \leq (m/a) + 1, \quad n/b \leq n_j \leq (n/b) + 1, \quad p/c \leq p_k \leq (p/c) + 1$$

for all $i \leq a$, $j \leq b$ and $k \leq c$. In other words, the best subdivisions are balanced.

Based on this fact, we decided to conduct the search in this interval for larger cases, to reduce the search space of the brute force approach. Thus, we used the following algorithm to determine subdivisions.

**Algorithm 3** `Subdivision`

**Input:**

- a product size $(m, n, p)$,
- a pattern size $(a, b, c)$

**Output:** subdivisions of $(m, n, p)$

1: base = $\lfloor m/a \rfloor$
2: rem = $m$ mod $a$
3: **for** $i = 1, \ldots, a$ **do**
4: $\quad \text{div}_m[\text{i}] = \text{base};$
5: **end for**
6: **for** $i = 1, \ldots, \text{rem}$ **do**
7: $\quad \text{div}_m[\text{j}] = \text{div}_m[\text{j}]+1;$
8: **end for**
9: Let $\text{D}_m$ be obtained as all permutations of $\text{div}_m$, removing duplicates
10: Do the same for $n$ and $p$. For $n$, use $b$ as divisor and $\text{div}_n$ and $\text{D}_n$ as arrays. For $p$, use $c$ as divisor and $\text{div}_p$ and $\text{D}_p$ as arrays.
11: return the list of all $(m_i, n_j, p_k)$ for $m_i$ in $\text{D}_m$, $n_j$ in $\text{D}_n$ and $p_k$ in $\text{D}_p$

### 7.2.4 Tracker

Tracker is the main program; it incrementally fills the database. Given a product size $(m, n, p)$, we will determine which pattern "fits it best", assuming that we have already figured out this information for all smaller sizes. We use all patterns in the list given Subsection 7.2.2, and the subdivision given in the previous subsection.

After a pattern and a subdivision have been chosen, and the dimensions of the recursive calls have been determined by `Optimizer`, we can either look up in the table for the costs of these recursive calls, or do some pairings. In the latter case, we say that two problem sizes $(m, n, p)$ and $(m', n', p')$ *can be paired* if either $m' = n, n' = p, p' = m$ or $m = n', n = p', p = m'$. In this case, the algorithm for two disjoint products of Section 3.2.6 can be used, for a cost of $K(m, n, p) = mnp + mn + np + pm$. We could use a similar strategy to compute three disjoint products, but it does not pay off in our sizes.

Finally, the minimum of all the possible costs is taken, and compared to the cost of the naive algorithm, and to the cost $A(n)$ of the algorithm of Chapter 5 if the product to perform is square.

**Algorithm 4** `Tracker`

**Input:**

- a product size $(m, n, p)$
- a list $\mathsf{L}$ of patterns
- a 3-dimensional table $\mathsf{T}$, giving the costs for all sizes smaller than $(m, n, p)$

let $t = mnp$
if $m = n = p$, then $t = \min(t, A(n))$
**for** each pattern $P$ in $\mathsf{L}$ **do**
  let $S = \texttt{Subdivision}((m, n, p), P)$
  **for** each subdivision $(\mathbf{m}, \mathbf{n}, \mathbf{p})$ in $S$ **do**
    let $(\mu_k, \nu_k, \pi_k)_{k \leq L} = \texttt{Optimizer}((m, n, p), P, (\mathbf{m}, \mathbf{n}, \mathbf{p}))$
    let $c = \sum_{k \leq L} \mathsf{T}[\mu_k, \nu_k, \pi_k]$
    for all pairs $k, k'$ of subproducts not already done, determine whether $(\mu_k, \nu_k, \pi_k)$ and $(\mu_{k'}, \nu_{k'}, \pi_{k'})$ can be paired, and whether the resulting cost $K(\mu_k, \nu_k, \pi_k)$ is lower than $\mathsf{T}[\mu_k, \nu_k, \pi_k] + \mathsf{T}[\mu_{k'}, \nu_{k'}, \pi_{k'}]$. If so, mark $k, k'$ as done and let $c = c - \mathsf{T}[\mu_k, \nu_k, \pi_k] - \mathsf{T}[\mu_{k'}, \nu_{k'}, \pi_{k'}] + K(\mu_k, \nu_k, \pi_k)$
    $t = \min(t, c)$
  **end for**
**end for**
return $t$

The information we get from the tracker is stored in a file in a format which actually contains more than just the number of multiplications: we also store the pattern and subdivision used, how many times the pairing method can be applied and finally the recursive call indexes where we may apply the pairing method. The implementation, in C, consists of following subprograms:

- **count.c** - acts as the base to build the database. Its task is to provide **controller.c** a pattern from a pattern list and the problem size $\langle m, n, p \rangle$.
- **controller.c** - after getting a pattern and the problem size, it will determine the minimum number of multiplications required and return the value to **count.c**
- **subdivision.c** - implements the algorithm `Subdivision`.
- **optimizer.c** - implements the algorithm `Optimizer`.

- **optmul.c** - determines the number of multiplications for the subproblems and returns the total calculated value of multiplications to **controller.c**.
- **pairs.c** - determines the possible subproblems where technique of two disjoint products can be applied, determines the number of multiplications and returns the value to **controller.c**.

## 7.3 Upper bounds

We determined upper bounds for matrix multiplication problems sizes up to $30 \times 30 \times 30$. The upper bounds we get are listed in the following table. We were able to improve the multiplication counts for most matrix sizes from 7 to 30; the entries in bold improve on the previous records. The gains are usually by a few dozens of multiplications. This should not be overlooked, since these problems have already a lot of previous attention.

| Dimension | Muls | Pattern Name | Probert-Fischer (1980) | Smith (2002) |
|---|---|---|---|---|
| 2 2 2 | 7 | Strassen | 7 | 7 |
| 3 3 3 | 23 | Laderman | 23 | 23 |
| 4 4 4 | 49 | Strassen | 49 | 49 |
| 5 5 5 | 100 | Makarov | 103 | 100 |
| 6 6 6 | 161 | Strassen | 161 | 161 |
| 7 7 7 | **258** | Strassen-Winograd | 276 | 273 |
| 8 8 8 | 343 | Strassen | 343 | 343 |
| 9 9 9 | **522** | Hopcroft323 | 529 | 529 |
| 10 10 10 | 700 | Strassen | 710 | 700 |
| 11 11 11 | **923** | Strassen | 996 | 992 |
| 12 12 12 | 1127 | Strassen | 1125 | 1125 |
| 13 13 13 | **1450** | Strassen | 1594 | 1580 |
| 14 14 14 | **1728** | Strassen | 1792 | 1778 |
| 15 15 15 | **2116** | Strassen-Winograd | 2369 | 2300 |
| 16 16 16 | 2401 | Strassen | 2401 | 2401 |
| 17 17 17 | **2972** | Strassen | 3218 | 3218 |
| 18 18 18 | **3306** | Chapter 5 | 3375 | 3342 |
| 19 19 19 | **4073** | Strassen | 4402 | 4369 |
| 20 20 20 | **4340** | Chapter 5 | 4870 | 4380 |
| 21 21 21 | **5365** | Strassen | 6131 | 5610 |
| 22 22 22 | **5566** | Chapter 5 | 6380 | 5610 |
| 23 23 23 | **6806** | Chapter 5 | 7875 | 7048 |
| 24 24 24 | **7000** | Chapter 5 | 7875 | 7048 |
| 25 25 25 | **8448** | Chapter 5 | 9676 | 8710 |
| 26 26 26 | **8658** | Chapter 5 | 9880 | 8710 |
| 27 27 27 | **10330** | Chapter 5 | 11984 | 10612 |
| 28 28 28 | **10556** | Chapter 5 | 11984 | 10612 |
| 29 29 29 | **12468** | Chapter 5 | 14360 | not listed |
| 30 30 30 | **12710** | Chapter 5 | 14360 | not listed |

Table 7.1: Upper bounds on the number of multiplications, non-commutative case

Remarks:

- For small powers of 2, as could be expected, no combination outperforms Strassen's algorithm.

- Even though Strassen's and Winograd's patterns both perform 7 multiplications, the difference in sparseness make them nonequivalent for our purposes: most of the time, Strassen's pattern is better, but for sizes 7 and 15, Winograd takes the lead.

- The algorithms of Laderman and Makarov are used only for the base cases (resp. 3 and 5); their relatively high exponents seem to make them useless for larger sizes.

- For sizes from 20 on, the algorithm of Chapter 5 takes a clear lead. This was already the case in the tables of [22, 26], and is supported by estimates of the exponents that would be obtained by applying this algorithm recursively. These exponents usually improve Strassen's; the optimal is obtained for $n = 44$, giving $\log_{44}(36300) \approx 2.775$.

The following table gives results for the case of matrices with commutative entries. In this case, an extra possibility is considered, using Waksman's algorithm presented in Chapter 6. Our results are compared to Mezzarobba's, who gave a similar table in [20].

| Dimension | Muls | Pattern Name | Mezzarobba (2007) |
|---|---|---|---|
| 2 2 2 | 7 | Strassen | 7 |
| 3 3 3 | **22** | makarov333 | 23 |
| 4 4 4 | 46 | Waksman | 46 |
| 5 5 5 | 93 | Waksman | 93 |
| 6 6 6 | 141 | Waksman | 141 |
| 7 7 7 | 235 | Waksman | 235 |
| 8 8 8 | 316 | Waksman | 316 |
| 9 9 9 | **472** | add121 | 473 |
| 10 10 10 | 595 | Waksman | 595 |
| 11 11 11 | **825** | add121 | 831 |
| 12 12 12 | 987 | Strassen | 987 |
| 13 13 13 | **1319** | add121 | 1333 |
| 14 14 14 | 1561 | Waksman | 1561 |
| 15 15 15 | **1953** | add121 | 2003 |
| 16 16 16 | 2212 | Strassen | 2212 |
| 17 17 17 | **2762** | Hopcroft332 | 2865 |
| 18 18 18 | **3060** | Hopcroft332 | 3231 |
| 19 19 19 | **3769** | Strassen | 3943 |
| 20 20 20 | 4165 | Strassen | 4165 |
| 21 21 21 | **5028** | Strassen | 5261 |
| 22 22 22 | **5566** | Chapter 5 | 5610 |
| 23 23 23 | **6386** | Hopcroft332 | 6843 |
| 24 24 24 | **6900** | Hopcroft332 | 6909 |
| 25 25 25 | **8085** | add121 | 8710 |
| 26 26 26 | **8658** | Chapter 5 | 8710 |
| 27 27 27 | **10118** | Strassen | 10612 |
| 28 28 28 | **10556** | Chapter 5 | 10612 |
| 29 29 29 | **12228** | Hopcroft332 | not listed |
| 30 30 30 | **12710** | Chapter 5 | not listed |

Table 7.2: Upper bounds on the number of multiplications, commutative case

# Chapter 8

# Code generation

This final chapter gives an overview of the techniques used in our implementation, the external packages we used, and reports on practical performance evaluations.

## 8.1 Introduction

Code generation simply refers to the work of generating code automatically to build some package or library for specific use. Indeed, to implement our techniques and evaluate their performance we have to first build up the database, then develop a program which generates individual multiplication procedures for problem sizes up to $30 \times 30 \times 30$, relying on predefined functions for addition, multiplication, etc.

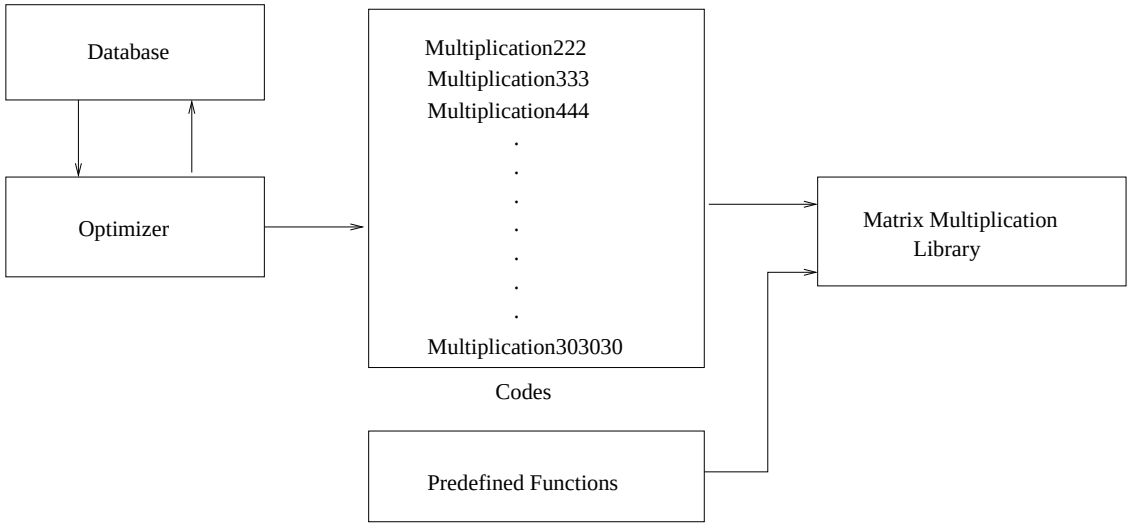



Figure 8.1: Block Diagram of Multiplication Function Generator

The target implementation language was either C or C++. We developed the

generating code in such a way that it was able to generate multiplication procedures for the following base rings:

- multiprecision integers
- univariate polynomials
- differential operator with polynomial coefficients
- linear recurrence operator with polynomial coefficients.

The first two examples are of obvious interest (applications were described in the introduction). The last ones arise in more specialized contexts.

## 8.2 Base ring arithmetic

### 8.2.1 Integers and polynomials

Arithmetic operations on integers and polynomials are well-documented [11], and efficient libraries exist. For the case of multiprecision integers, we use the GMP library [1] to provide basic functionalities; for polynomials, we use the NTL C++ package [25].

### 8.2.2 Differential operators

The arithmetic of differential operators is less straightforward than that of integers or polynomials. A differential operator $a$ is an expression

$$a = \sum_{i=0}^{d} a_i(x)\partial^i,$$

where $a_i(x)$ are polynomials in a variable $x$, and $\partial$ stands for the operator $\partial/\partial x$. Thus, $a$ represents the operator

$$f \mapsto a(f) = \sum_{i=0}^{d} a_i(x)\frac{\partial^i f}{\partial x^i},$$

where $f$ is (for instance) a polynomial in $x$. For instance, the operator $a = x$ is multiplication-by-$x$, with $a(f) = xf$; the operator $b = \partial$ is differentiation-with-respect-to-$x$, with $b(f) = \partial f/\partial x$.

Addition and subtractions are defined easily, coefficient-wise. The multiplication of operator describes composition: $c = ab$ is such that $c(f) = a(b(f)) = x\partial f/\partial x$. Hence, the product $c = ab$ can be written as $c = x\partial$. On the other hand, the product $e = ba$ is such that $e(f) = b(a(f)) = \partial(xf)/\partial x$. Differentiation rules show that $e(f) = x\partial f/\partial x + f$, so that $e = ba$ can be rewritten as $x\partial + 1$. Working out the details in the general case gives

$$\sum_{i\leq d} a_i(x)\partial^i \;\times\; \sum_{j\leq e} b_j(x)\partial^j \;=\; \sum_{i\leq d} a_i(x) \sum_{j\leq e}\sum_{k\leq i} \binom{i}{k} b_j(x)^{(k)}\partial^{i+j-k}, \qquad (8.1)$$

where $b_j(x)^{(k)}$ is the $k$th derivative of $b_j$. This gives rises to a non-commutative ring. Many algorithms exist for differential operators, including linear algebra algorithms [7]; this justifies our interest in developing efficient matrix multiplication for these objects.

Pseudo-code implementing the previous formula is given below. Differential operators are stored as arrays of polynomials, and we assume that basic operations on polynomials are available; for simplicity, we allow (at step 12) to multiply all entries of an array by a same polynomial, and to add arrays.

**Algorithm 5** `DiffMultiplication`

**Input:** differential operators $a = \sum_{i\leq d} a_i(x)\partial^i$ and $b = \sum_{i\leq e} b_i(x)\partial^i$,

**Output:** $c = ab$

1: let $c$ and $c'$ be arrays of length $i + j + 1$
2: **for** $i = 0, \ldots, d$ **do**
3: set all entries of $c'$ to zero
4: **for** $j = 0, \ldots, e$ **do**
5: $t = b[j]$
6: **for** $k = 0, \ldots, i$ **do**
7: $u = \text{binomial}(i, k)t$
8: $c'[i + j - k] = c'[i + j - k] + u$
9: $t = t'$
10: **end for**
11: **end for**
12: $c = c + a[i]c'$
13: **end for**
14: return $c$

### 8.2.3 Linear recurrences

Linear recurrences are a discrete analogue of differential equations. A linear recurrence $a$ is an expression

$$a = \sum_{i=0}^{d} a_i(n) E^i,$$

where $a_i(n)$ are polynomials in the variable $n$, and $E$ stands for the *shift operator*. The latter is defined as taking a sequence $u = (u_0, u_1, \dots)$ as input, and returning $E(u) = (u_2, u_2, \dots)$. More generally, the recurrence $a$ is such that

$$a(u) = (v_0, v_1, \dots) \quad \text{with} \quad v_n = \sum_{i=0}^{d} a_i(n) u_{n+i}.$$

For instance, the operator $a = n$ is multiplication-by-$n$ and the operator $b = E$ is shifting by one place. As for differential operator, linear recurrences can be multiplied. We obtain on these examples

$$ab = nE \quad \text{and} \quad ba = (n+1)E,$$

and in general

$$\sum_{i \le d} a_i(n) E^i \;\times\; \sum_{j \le e} b_j(n) E^j \;=\; \sum_{i \le d} a_i(n) \sum_{j \le e} b_j(n+i) E^{i+j}. \tag{8.2}$$

This gives another non-commutative ring. As for differential operators, algorithms for linear recurrences have been thoroughly studied. Actually, both linear recurrences and differential operators are members of a large class of algebraic structures known as Ore rings, and questions such as linear algebra algorithms studied in [7] are valid for the whole class of Ore rings.

In the following pseudo-code, we use the same conventions as for the differential operator case. Given a polynomial $p(n)$, the function shift$(p, i)$ returns the polynomial $p(n+i)$; it is implemented in a straightforward manner.

**Algorithm 6** `RecurrenceMultiplication`

**Input:** linear recurrences $a = \sum_{i \leq d} a_i(n)E^i$ and $b = \sum_{i \leq e} b_i(n)E^i$,

**Output:** $c = ab$

1: let $c$ and $c'$ be arrays of length $i + j + 1$
2: **for** $i = 0, \ldots, d$ **do**
3: set all entries of $c'$ to zero
4: **for** $j = 0, \ldots, e$ **do**
5: $c'[i + j - k] = c'[i + j - k] + \text{shift}(b[j], i)$
6: **end for**
7: $c = c + a[i]c'$
8: **end for**
9: return $c$

## 8.3 Code generation

To generate multiplication function code for the application families which we consider, we developed a tool in C, which itself produces C or C++ code. The C code implements the integer version of our algorithms, and uses GMP data structures and basic functions; the C++ version allows us to use the NTL library for polynomial arithmetic, and supports the polynomial case, as well as differential operators and linear recurrences.

The code generator was developed as a proof-of-concept, with a view toward simplicity. Thus, it does not use mechanisms such as C++ template programming; as a result, there is one version of multiplication code per supported base ring. As input, it takes the following:

- the pattern files, already used in the previous chapter,
- the database produced by the optimization process of the previous chapter,
- user-defined functions for operations in the target base rings.

Also, we rely on predefined functions for two disjoint product computation, and the algorithms of Chapters 5 and 6. As output, in the generated code, we have one function for each square problem size between 2 and 30, as well as for all needed subproblem sizes.

The code generator does not optimize the linear combinations, which are done in a naive manner. This is apparent on the following example, which shows the

multiplication code generated for multiprecision integers in size $\langle 2, 2, 2\rangle$. In this case, our code generator uses Strassen's pattern.

```
void Multiplication222(mpz_t *A,mpz_t *B,mpz_t *C)
{
mpz_t *p,*S1,*S2;
int t1,t2;
S1=(mpz_t*) malloc(1*sizeof(mpz_t));
S2=(mpz_t*) malloc(1*sizeof(mpz_t));
p=(mpz_t*) malloc(1*sizeof(mpz_t));
init(S1,1);
init(S2,1);
init(p,1);
set_zero(S1,1);
set_zero(S2,1);
set_zero(S1,1);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 1, 1);
subsubmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 0, 0);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 1, 0);
naive(S1, S2, 1, 1, 1, 1, p, 1, 1);
Caddmatrix(p,1,1,C,1,1,C,1,1,2,0,0);
Caddmatrix(p,1,1,C,1,1,C,1,1,2,1,0);
set_zero(S1,1);
set_zero(S2,1);
set_zero(S1,1);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 0, 0);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 0, 1);
subsubmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 1, 1);
naive(S1, S2, 1, 1, 1, 1, p, 1, 1);
set_zero(S1,1);
set_zero(S2,1);
set_zero(S1,1);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 1, 0);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 1, 1);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 0, 0);
naive(S1, S2, 1, 1, 1, 1, p, 1, 1);
Caddmatrix(p,1,1,C,1,1,C,1,1,2,1,0);
```

```
Csubmatrix(C,1,1,p,1,1,C,1,1,2,1,1);
set_zero(S1,1);
set_zero(S2,1);
set_zero(S1,1);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 0, 0);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 0, 1);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 1, 1);
naive(S1, S2, 1, 1, 1, 1, p, 1, 1);
Csubmatrix(C,1,1,p,1,1,C,1,1,2,0,0);
Caddmatrix(p,1,1,C,1,1,C,1,1,2,0,1);
set_zero(S1,1);
set_zero(S2,1);
set_zero(S1,1);
subsubmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 0, 0);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 1, 0);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 0, 0);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 0, 1);
naive(S1, S2, 1, 1, 1, 1, p, 1, 1);
Caddmatrix(p,1,1,C,1,1,C,1,1,2,1,1);
set_zero(S1,1);
set_zero(S2,1);
set_zero(S1,1);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 0, 1);
subsubmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 1, 1);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 1, 0);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 1, 1);
Caddmatrix(p,1,1,C,1,1,C,1,1,2,0,0);
set_zero(S1,1);
set_zero(S2,1);
set_zero(S1,1);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 0, 0);
subaddmatrix(S1, 1, 1, A, 1, 1, S1, 1, 1, 2, 1, 1);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 0, 0);
subaddmatrix(S2, 1, 1, B, 1, 1, S2, 1, 1, 2, 1, 1);
naive(S1, S2, 1, 1, 1, 1, p, 1, 1);
Caddmatrix(p,1,1,C,1,1,C,1,1,2,0,0);
```

```
Caddmatrix(p,1,1,C,1,1,C,1,1,2,1,1);
clear(S1,1);
clear(S2,1);
clear(p,1);
free(S1);
free(S2);
free(p);
}
```

## 8.4 Experimental results

All timings and results for the following benchmarks are obtained using an INTEL Core 2 Duo(64 bit) machine, with CPU speed 2.4 GHz and 3 GB of RAM. All timings in Section 8.4.1 are averaged over 150 runs; those in Section 8.4.2 are averaged over 10 runs.

### 8.4.1 Commutative entries

The following graphs give timings for integers with respective lengths 1000 and 10000 bits. For the smaller integers, it appears that a low multiplication counts does not imply a faster running-time, since in particular we must perform more additions. For larger entries, our approach becomes better, as expected. Clearly, a more refined study is needed for the case of matrices with small entries.

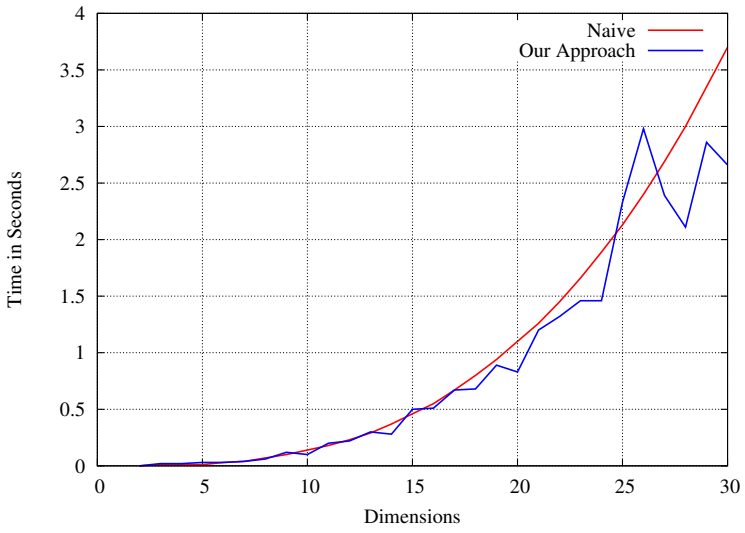



Figure 8.2: Performance for integer entries (length 1000 bit)

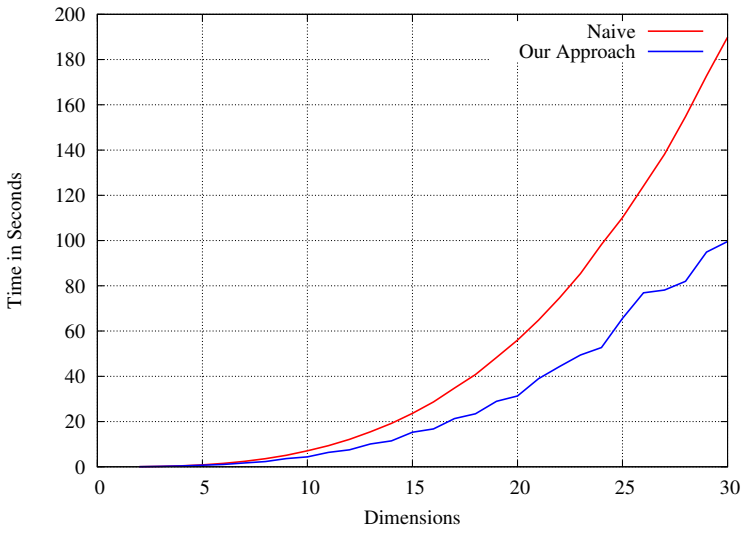



Figure 8.3: Performance for integer entries (length 10000 bit)

The next graphs give timings for polynomials with respective degree 100 and 400, with coefficients over a small finite field. In this case, our approach is significantly better for both ranges. We use NTL C++ package [25] which provides us with the facilities of declaring polynomials and built-in arithmetic functions for polynomials.

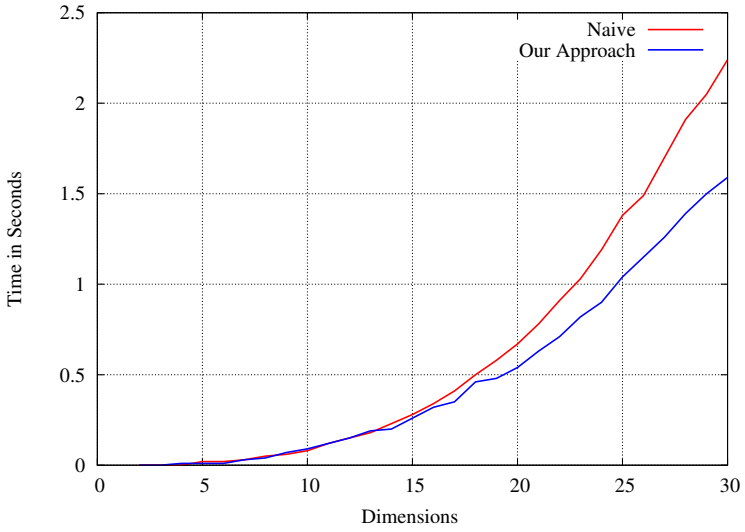



Figure 8.4: Performance for polynomial entries (degree 100)

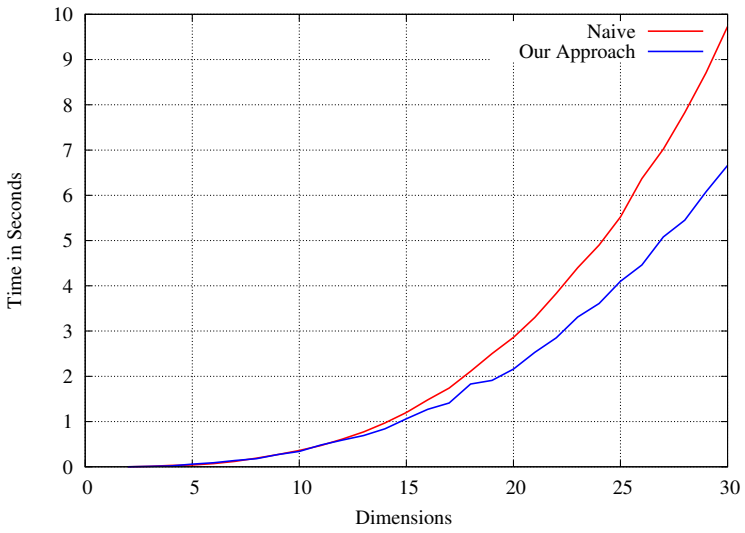



Figure 8.5: Performance for polynomial entries (degree 400)

It is interesting to note that for commutative cases, Waksman's algorithm is already quite useful by itself. This is all the more the case as our implementation optimizes the number of additions and subtractions, which play a non-negligible role for not-huge integers. The following graph shows the relative performance of our approach vs. Waksman's algorithm, on integers of 10000 bits.

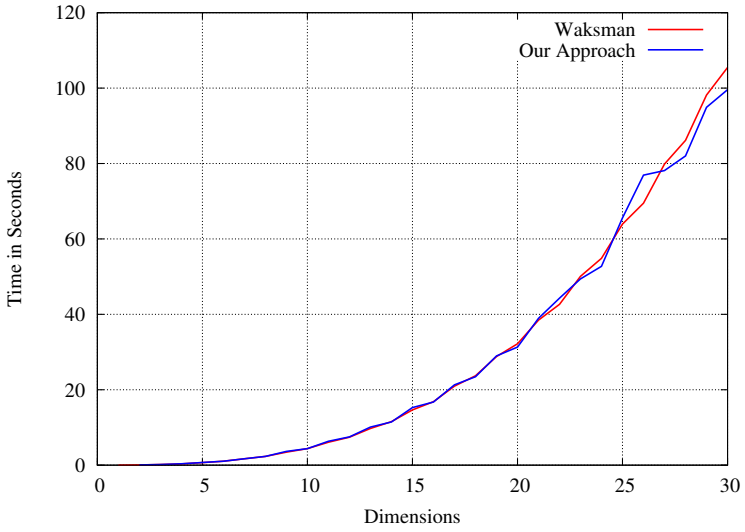



Figure 8.6: Waksman vs. our algorithm (length 10000 bit)

Finally, we compare our integer multiplication to the one in Maple 11. We use the function `LinearAlgebra:-LA_Main:-MatrixMatrixMultiply`, to remove some of the overhead that would be induced by calling `LinearAlgebra:-MatrixMatrixMultiply`.

Our code performs much better, but it is hardly a surprise: further profiling would be needed to understand where the time is spent in the Maple version.

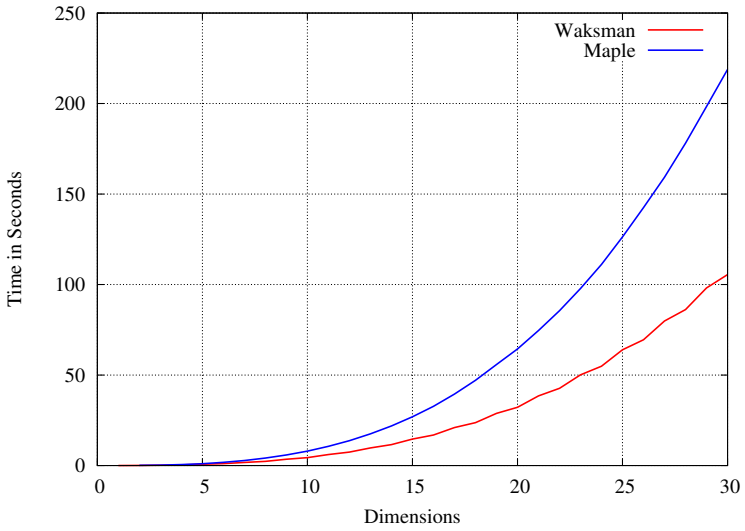



Figure 8.7: Waksman vs. Maple (length 10000 bit)

### 8.4.2 Non-commutative entries

In our next example, the entries of the matrices are differential operators of order 10, with polynomial coefficients of degree 10, over a small finite field. Our approach is clearly better on such cases, since the cost of multiplication is much higher than that of additions, so that saving multiplications pays off quickly. Indeed, the running-time curves closely match the number of multiplications that are performed. Recall that in this case, Waksman's algorithm is not applicable.

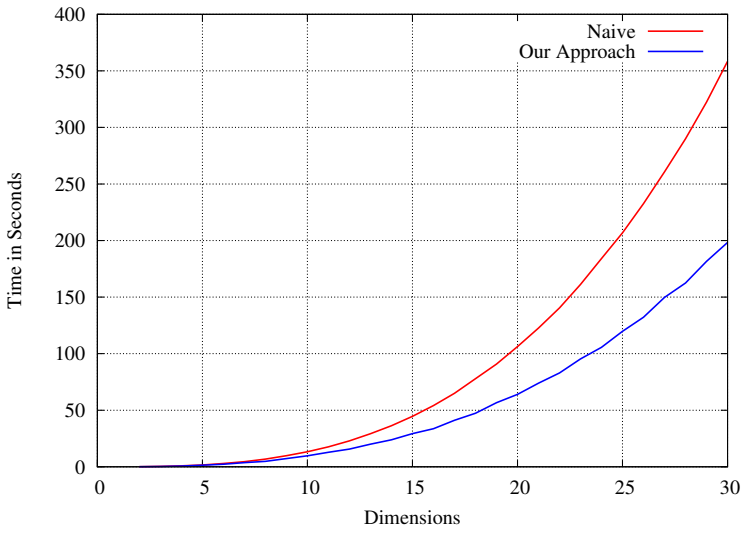



Figure 8.8: Performance for differential operator entries

The final graph shows the results for linear recurrences. The conclusion is similar: when the cost of multiplication is high compared to additions, it is obviously useful to reduce the number of multiplications.

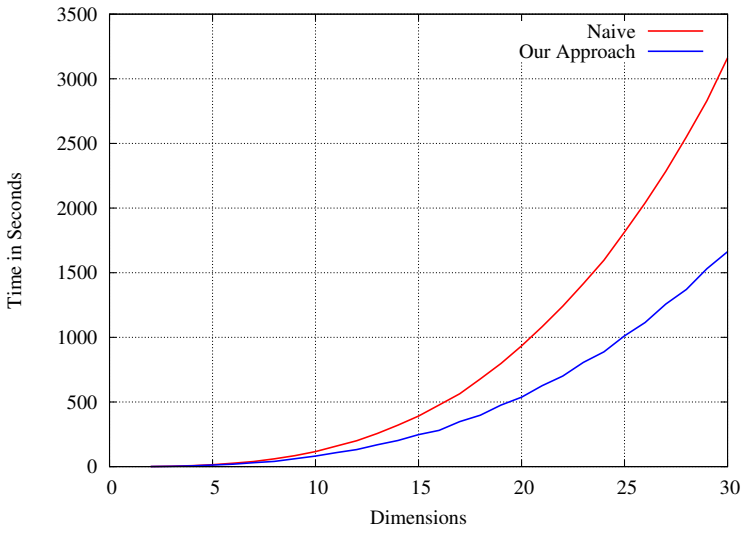



Figure 8.9: Performance for linear recurrence entries

# Chapter 9

# Conclusion

There are many ways in which this research may be enhanced or expanded.

- To lower further the number of multiplications, one may consider modifying the given patterns, by trying to make them sparser. The rules allowed to modify patterns are well-known [26], and may be amenable to automated search.
- Our techniques may be used for matrices with sparse structure, with adequate modification. This may be useful for algorithms for holonomic function evaluation [20], which feature some sparse matrices.
- The code generator must be optimized, with respect to speed and memory management. In particular, more care should be taken to handle the linear combination steps, instead of our naive design.
- The code generator should also be modified towards more genericity (such as producing templated C++ code, to be used in libraries such as LinBox), and support more target languages, such as Aldor.

# Bibliography


[1] GMP: Arithmetic without limitations. `http://gmplib.org`.

[2] B. Beckermann and G. Labahn. A uniform approach for the fast computation of matrix-type Padé approximants. *SIAM J. Matrix Analysis and Applic.*, 15(3):804–823, 1994.

[3] D. Bini, M. Capovani, F. Romani, and G. Lotti. $O(n^{2.7799})$ complexity for $n \times n$ approximate matrix multiplication. *Inf. Proc. Lett.*, 8(5):234–235, 1979.

[4] M. Bläser. A $5/2n^2$-lower bound for the rank of $n \times n$ matrix multiplication over arbitrary fields. In *FOCS'99*, pages 45–50. IEEE, 1999.

[5] M. Bläser. A $5/2n^2$-lower bound for the multiplicative complexity of $n \times n$-matrix multiplication. In *STACS'01*, volume 2010 of *Lecture Notes in Computer Science*, pages 99–109. Springer, 2001.

[6] P. Bürgisser, M. Clausen, and M. A. Shokrollahi. *Algebraic complexity theory*, volume 315 of *Grund. der Math. Wiss.* Springer, 1997.

[7] H. Cheng. *Algorithms for Normal Forms for Matrices of Polynomials and Ore Polynomials*. PhD thesis, University of Waterloo, 2003.

[8] H. Cohn, R. D. Kleinberg, B. Szegedy, and C. Umans. Group-theoretic algorithms for matrix multiplication. In *FOCS'05*, pages 379–388. IEEE, 2005.

[9] H. Cohn and C. Umans. A group-theoretic approach to fast matrix multiplication. In *FOCS'03*, pages 438–449. IEEE, 2003.

[10] D. Coppersmith and S. Winograd. Matrix multiplication via arithmetic progressions. *Journal of Symbolic Computation*, 9(3):251–280, 1990.

[11] J. von zur Gathen and J. Gerhard. *Modern Computer Algebra*. Cambridge University Press, 1999.

[12] J. Hopcroft and J. Musinski. Duality applied to the complexity of matrix multiplications and other bilinear forms. In *STOC'73*, pages 73–87. ACM, 1973.

[13] J. E. Hopcroft and L. R. Kerr. On minimizing the number of multiplications necessary for matrix multiplication. *SIAM Journal on Applied Math.*, 20(1):30–36, 1971.

[14] I. Kaporin. A practical algorithm for faster matrix multiplication. *Numerical Linear Algebra with Applications*, 6:687–700, 1999.

[15] I. Kaporin. The aggregation and cancellation techniques as a practical tool for faster matrix multiplication. *Theor. Comput. Sci.*, 315(2-3):469–510, 2004.

[16] J. Laderman. A noncommutative algorithm for multiplying $3 \times 3$ matrices using 23 multiplications. *Bull. Amer. Math. Soc.*, 82:126–128, 1976.

[17] J. Laderman, V. Pan, and X. H. Sha. On practical algorithms for accelerated matrix multiplication. *Linear Algebra Appl.*, 162/164:557–588, 1992.

[18] O. M. Makarov. An algorithm for multiplication of $3 \times 3$ matrices. *Zh. Vychisl. Mat. i Mat. Fiz.*, 26(2):293–294, 320, 1986.

[19] O. M. Makarov. A noncommutative algorithm for multiplying $5 \times 5$ matrices using one hundred multiplications. *U.S.S.R. Comput. Maths. Phys.*, 27:205–207, 1987.

[20] M. Mezzarobba. Génération automatique de procédures numériques pour les fonctions D-finies. Master's thesis, Master parisien de recherche en informatique, 2007.

[21] V. Pan. How can we speed up matrix multiplication? *SIAM Rev.*, 26(3):393–415, 1984.

[22] R. L. Probert and P. C. Fischer. Decomposition techniques for matrix multiplication problems. *Utilitas Mathematica*, 18:257–267, 1980.

[23] A. Schönhage. Partial and total matrix multiplication. *SIAM J. Comput.*, 10(3):434–455, 1981.

[24] É. Schost. Complexity results for triangular sets. *J. Symb. Comput.*, 36(3-4):555–594, 2003.

[25] V. Shoup. NTL: A library for doing number theory. `http://www.shoup.net`.

[26] W. D. Smith. Fast matrix multiplication formulae-report of the prospectors, 2002.

[27] V. Strassen. Gaussian elimination is not optimal. *Numerische Mathematik*, 13(4):354–356, 1969.

[28] V. Strassen. The asymptotic spectrum of tensors. *Journal für die reine und angewandte Mathematik*, 384:102–152, 1988.

[29] A. Waksman. On Winograd's algorithm for inner products. *IEEE Transactions On Computers*, pages 360–361, April 1970.

[30] S. Winograd. On the algebraic complexity of inner product. *Linear Algebra and Appl.*, 4:377–379, 1971.

[31] S. Winograd. Some remarks on fast multiplication of polynomials. In *Complexity of sequential and parallel numerical algorithms*, pages 181–196. Academic Press, 1973.

# Curriculum Vitae

| | |
|---|---|
| **Name**: | **Md. Nazrul Islam** |
| **Post-Secondary Education and Degrees**: | The University of Western Ontario<br>London, Ontario, Canada<br>M.Sc. Computer Science, August 2009<br><br>Khulna University of Engineering & Technology (KUET)<br>Khulna, Bangladesh<br>B.Sc. in Computer Science and Engineering Mar. 2005 |
| **Work Experience**: | Research Assistant, Teaching Assistant.<br>University of western Ontario, London, Canada.<br>Sept. 2008 - August 2009<br><br>Full time faculty member<br>Dept. of computer science & eng., KUET, Bangladesh.<br>May 2006 - August 2008<br><br>Part time faculty member<br>Dept. of computer science & eng., KUET, Bangladesh.<br>March 2006 - April 2006<br><br>Part time faculty member<br>Dept. of computer science & eng..<br>Military Institute of Science and Technology, Bangladesh.<br>Dec. 2005 - March 2006<br><br>Asst. System Administrator<br>United Group, Bangladesh<br>April 2005 - Sep. 2005 |